

软件安全作业二 —— 思考题

班级：2022211801 学号：2022211570 姓名：项枫

一、测试

1、CWE121_Stack_Based_Buffer_Overflow__CWE805_wchar_t_declare_snprintf_05.c

测试代码：

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if(staticTrue)
    {
        data = dataBadBuffer;
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source);
        printWLine(data);
    }
}
```

运行结果：运行

test01/CWE121_Stack_Based_Buffer_Overflow__CWE805_wchar_t_declare_snprintf_05.c:

阅书万卷，我是你的知识宝典

超拟人数字人

新建群聊

新建智能体

讯飞星火

在这段代码中，存在一个潜在的缓冲区溢出漏洞。具体问题出现在以下几行：

```
c
SNPRINTF(data, 100, L"%s", source);
```

SNPRINTF 是一个宏，通常用于安全地格式化字符串并防止缓冲区溢出。然而，这里 data 指向的是 dataBadBuffer，其大小为 50 个 wchar_t，而 source 的大小为 100 个 wchar_t。当 SNPRINTF 试图将 source 的内容复制到 data 时，由于 data 的缓冲区大小不够大，会导致缓冲区溢出。

修复方法

为了修复这个漏洞，可以确保 data 指向的缓冲区足够大以容纳 source 的内容。可以将 data 指向 dataGoodBuffer，因为 dataGoodBuffer 的大小是 100，足以容纳 source 的内容。

以下是修复后的代码：

```
c
void function()
{
    data = dataGoodBuffer; // 修改为指向 dataGoodBuffer
    data[0] = L'\0';
    wchar_t source[100];
    wmemset(source, L'C', 100-1);
    source[100-1] = L'\0';
    SNPRINTF(data, 100, L"%s", source);
    printWLine(data);
}
```

代码纠错 内容写作 文本润色 网页摘要 中英翻译 学习计划 居家健身 更多 全新对话

点这里，尽管问，Shift+Enter 换行

内容由讯飞星火大模型生成，仅供参考

阅书万卷，我是你的知识宝典

超拟人数字人

新建群聊

新建智能体

void function()

```
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if(staticTrue)
    {
        data = dataGoodBuffer; // 修改为指向 dataGoodBuffer
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source);
        printWLine(data);
    }
}
```

这样修改后，data 指向的缓冲区大小为 100，能够安全地容纳 source 的内容，从而避免了缓冲区溢出的问题。

问问智能体: 智能编程助手iFlyCode 代码纠错小助手 代码注释添加助手

代码纠错 内容写作 文本润色 网页摘要 中英翻译 学习计划 居家健身 更多 全新对话

点这里，尽管问，Shift+Enter 换行

内容由讯飞星火大模型生成，仅供参考

结果分析：

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行：

```
SNPRINTF(data, 100, L"%s", source);
```

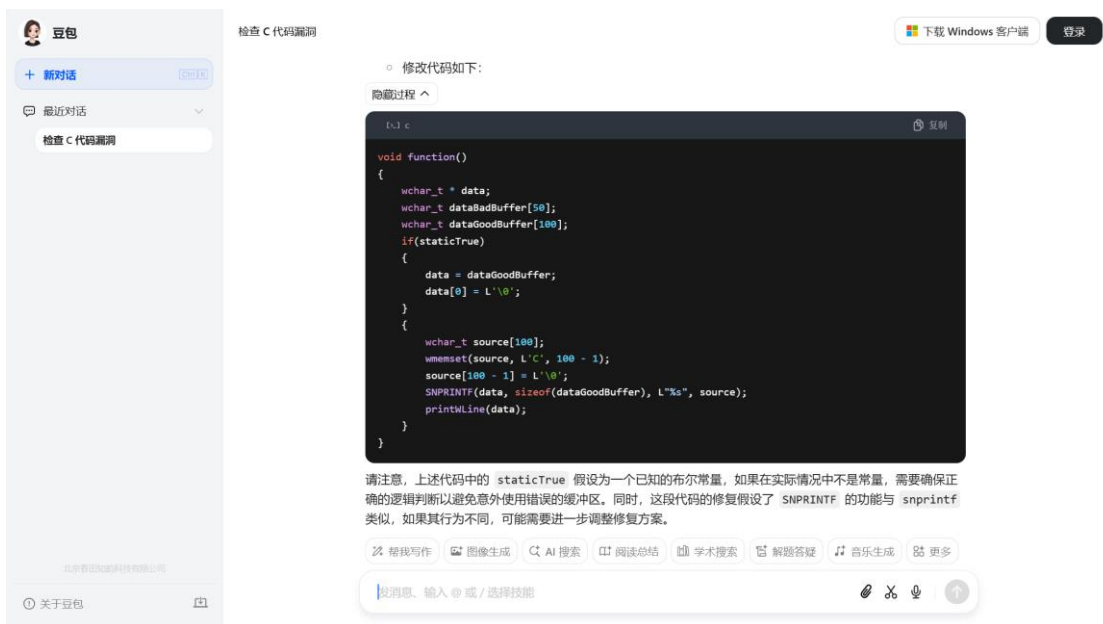
模型修改的代码：

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
```

3 / 113



测试输出：



结果分析：

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行：

```
SNPRINTF(data, 100, L"%s", source);
```

模型修改的代码：

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if(staticTrue)
    {
        data = dataGoodBuffer;
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100 - 1);
        source[100 - 1] = L'\0';
        SNPRINTF(data, sizeof(dataGoodBuffer), L"%s", source);
        printWLine(data);
    }
}
```

代码运行结果：运行 test01/doubao.c：

运行结果正确。

(3) ChatGPT

测试输入：



测试输出：

漏洞分析

1. 数据缓冲区溢出漏洞:

- 在以下代码中:

```
cpp
data = dataBadBuffer;
```

`data` 被设置为 `dataBadBuffer`, 其大小为 50.

- 然后, 在调用 `SNPRINTF` 时:

```
cpp
SNPRINTF(data, 100, L"%s", source);
```

这里尝试将 `source` (大小为 100) 写入 `data` (大小为 50), 这会导致缓冲区溢出.

- 在 `dataBadBuffer` 的大小是 50 字节时, 向 `data` 写入 100 字节的数据 (由 `source` 提供) 会导致缓冲区溢出, 从而可能引发未定义行为 (如覆盖其他内存、造成程序崩溃等).

2. 使用 `SNPRINTF` 函数时可能存在的风险:

- 即使在代码中已经设置了 `SNPRINTF` 的大小限制 (100), 但是如果 `data` 指向的是一个较小的缓冲区 (如 `dataBadBuffer`), 那么依然会发生缓冲区溢出.

修复建议

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

修复后的代码:

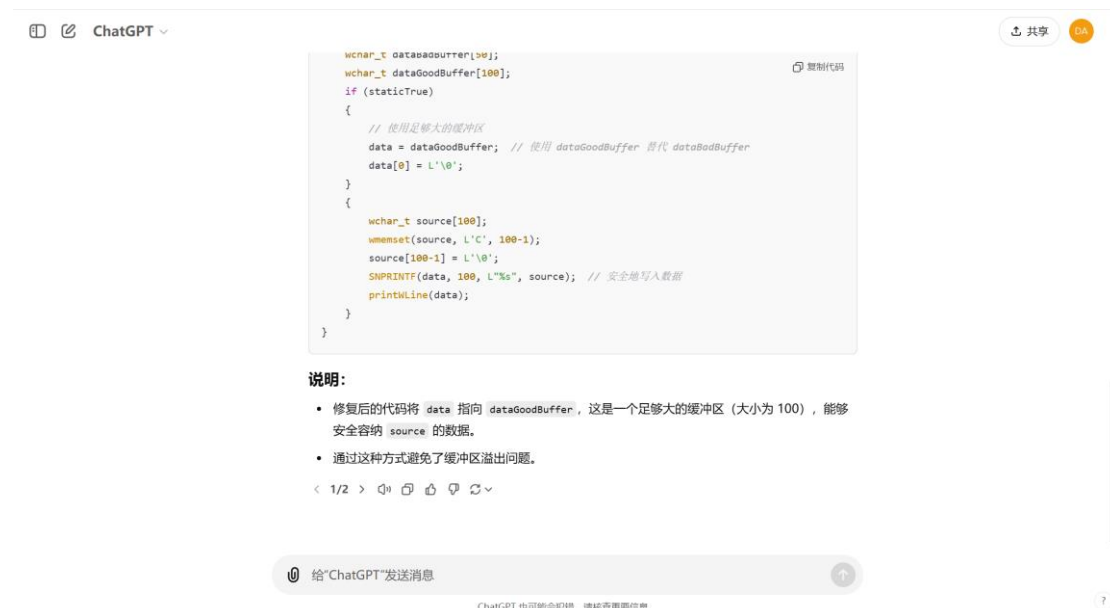
```
cpp
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if (staticTrue)
    {
        // 使用足够大的缓冲区
        data = dataGoodBuffer; // 使用 dataGoodBuffer 替代 dataBadBuffer
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source); // 安全地写入数据
        printLine(data);
    }
}
```

说明:

- 修复后的代码将 `data` 指向 `dataGoodBuffer`, 这是一个足够大的缓冲区 (大小为 100), 能够

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析:

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行:

```
data = dataBadBuffer;
SNPRINTF(data, 100, L"%s", source);
```

模型修改的代码:

```
void function()
{
    wchar_t * data;
    wchar_t dataBadBuffer[50];
    wchar_t dataGoodBuffer[100];
    if (staticTrue)
    {
        // 使用足够大的缓冲区
        data = dataGoodBuffer; // 使用 dataGoodBuffer 替代 dataBadBuffer
        data[0] = L'\0';
    }
    {
        wchar_t source[100];
        wmemset(source, L'C', 100-1);
        source[100-1] = L'\0';
        SNPRINTF(data, 100, L"%s", source); // 安全地写入数据
        printWLine(data);
    }
}
```

代码运行结果：运行 test01/chatgpt.c:

运行结果正确。

[illegible]

2、CWE78_OS_Command_Injection__char_file_w32_execlv_06.c

测试代码：

```
void function()
{
    char * data;

    char dataBuffer[100] = COMMAND_ARG2;

    data = dataBuffer;

    if (STATIC_CONST_FIVE==5)
    {
        {
            size_t dataLen = strlen(data);

            FILE * pFile;

            if (100-dataLen > 1)
            {
                pFile = fopen(FILENAME, "r");

                if (pFile != NULL)
                {
                    if (fgets(data+dataLen, (int)(100-dataLen), pFile) == NULL)
                    {
                        printLine("fgets() failed");

                        data[dataLen] = '\0';
                    }

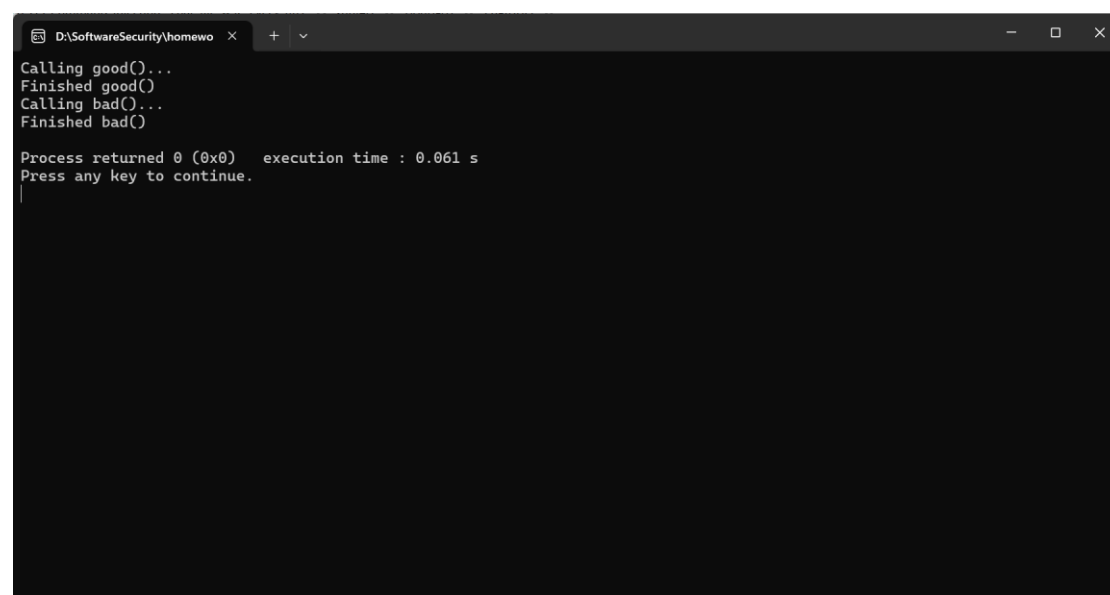
                    fclose(pFile);
                }
            }
        }
    }
}
```

```

    }
}
}
{
    char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
    EXECV(COMMAND_INT_PATH, args);
}
}

```

运行结果：运行 test02/ CWE78_OS_Command_Injection__char_file_w32_execv_06.c:



```

D:\SoftwareSecurity\homewo
Calling good()...
Finished good()
Calling bad()...
Finished bad()
Process returned 0 (0x0) execution time : 0.061 s
Press any key to continue.

```

(1) 讯飞星火

测试输入:



测试输出：





结果分析:

该模型找到了测试代码的缓冲区溢出、文件未关闭、命令注入风险漏洞并提供了漏洞所在行:

```
if (100-dataLen > 1)
fclose(pFile);
EXECV(COMMAND_INT_PATH, args);
```

模型修改的代码:

```
void function()
```

```

{
    char * data;
    char dataBuffer[100] = COMMAND_ARG2;
    data = dataBuffer;
    if (STATIC_CONST_FIVE==5)
    {
        {
            size_t dataLen = strlen(data);
            FILE * pFile;
            if (dataLen < 99) // 确保 dataLen 小于 99
            {
                pFile = fopen(FILENAME, "r");
                if (pFile != NULL)
                {
                    if (fgets(data+dataLen, (int)(100-dataLen), pFile) == NULL)
                    {
                        printLine("fgets() failed");
                        data[dataLen] = '\0';
                    }
                    fclose(pFile); // 确保文件被正确关闭
                }
            }
        }
    }
    {
        char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
        EXECV(COMMAND_INT_PATH, args); // 确保 COMMAND_ARG2 安全
    }
}

```

代码运行结果：运行 test02/xunfei.c:

运行结果正确。

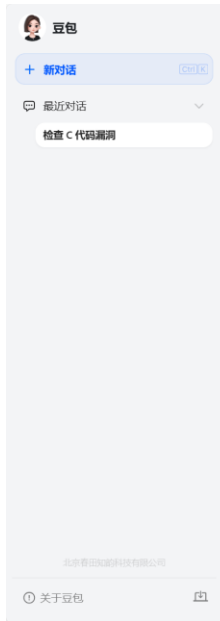
```
D:\SoftwareSecurity\homewo x + v - □ x
xunfei...
Process returned 0 (0x0)   execution time : 0.035 s
Press any key to continue.
```

(2) 豆包

测试输入：



测试输出：



检查 C 代码漏洞

下载 Windows 客户端

登录

以下是对这段代码的分析：

一、漏洞分析

1. 可能的资源泄露：

- 漏洞行号：在 `fgets` 失败的情况下，没有检查文件是否已经正确关闭，可能导致资源泄露。例如在 `if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)` 判断为真时，只执行了 `printf("fgets() failed");` 和 `data[dataLen] = '\0';`，没有关闭文件指针 `pFile`。
- 原因：如果 `fgets` 失败，文件可能没有被正确关闭，长期运行可能导致资源耗尽。

2. 命令注入风险：

- 漏洞行号：`char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};` 和 `EXECV(COMMAND_INT_PATH, args);`。
- 原因：如果 `COMMAND_ARG1` 和 `COMMAND_ARG3` 是由外部输入或者不可信来源构造的，可能存在命令注入风险。

二、修复建议

1. 防止资源泄露：

- 修改代码如下：

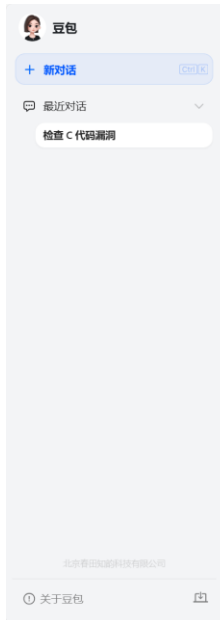
隐藏过程 ^

```
[...] c
if (100 - dataLen > 1)
{
    pFile = fopen(FILENAME, "r");
}
```

帮我写作 图像生成 AI 搜索 阅读总结 学术搜索 解答题疑 音乐生成 更多

点击登录，以便可以继续提问聊天

🔍 ✂️ 📄 ⬆️



检查 C 代码漏洞

下载 Windows 客户端

登录

```
[...] c
if (100 - dataLen > 1)
{
    pFile = fopen(FILENAME, "r");
    if (pFile != NULL)
    {
        if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
        {
            printf("fgets() failed");
            data[dataLen] = '\0';
            fclose(pFile);
        }
        else
        {
            fclose(pFile);
        }
    }
}
```

- ##### 2. 防止命令注入：
- 对 `COMMAND_ARG1` 和 `COMMAND_ARG3` 进行严格的输入验证和过滤，确保它们不包含任何可能被解释为命令的特殊字符。如果可能，尽量避免使用外部输入来构造命令行参数。或者使用更安全的执行命令的方式，避免直接使用 `execv`。

请注意，以上修复建议仅为示例，实际应用中还需要根据具体情况进行更详细的安全检查和调整。同时，代码中的 `COMMAND_ARG2` 直接赋给数组 `dataBuffer`，如果 `COMMAND_ARG2` 的长度超过 `dataBuffer` 的大小，也会导致缓冲区溢出风险，需要确保 `COMMAND_ARG2` 的长度合适或者进行适当的长度检查和处理。

帮我写作 图像生成 AI 搜索 阅读总结 学术搜索 解答题疑 音乐生成 更多

点击登录，以便可以继续提问聊天

🔍 ✂️ 📄 ⬆️

结果分析：

该模型找到了测试代码的资源泄露、命令注入漏洞并提供了漏洞所在行：

```
f (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
```

```
char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
EXECV(COMMAND_INT_PATH, args);
```

模型修改的代码：

```
if (100 - dataLen > 1)
{
    pFile = fopen(FILENAME, "r");
```

```

if (pFile!= NULL)
{
    if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
    {
        printLine("fgets() failed");
        data[dataLen] = '\0';
        fclose(pFile);
    }
    else
    {
        fclose(pFile);
    }
}
}
}

```

代码运行结果：运行 test02/doubao.c:

代码运行失败:

```

||=== Build file: "no target" in "no project" (compiler: unknown) ===|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c||In function 'function':|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c|48|error: 'dataLen' undeclared
(first use in this function); did you mean 'data'?|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c|48|note: each undeclared
identifier is reported only once for each function it appears in|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c|50|error: 'pFile' undeclared
(first use in this function)|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c|50|warning: unknown escape
sequence: '\S'|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c|50|warning: unknown escape
sequence: '\h'|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c|50|warning: unknown escape
sequence: '\h'|
D:\SoftwareSecurity\homework\hw2\test\test02\doubao.c|68|warning: passing argument 2 of
'_execv' from incompatible pointer type [-Wincompatible-pointer-types]|
C:\Program Files\CodeBlocks\MinGW\x86_64-w64-mingw32\include\process.h|66|note: expected
'const char * const*' but argument is of type 'char **'|
||=== Build failed: 2 error(s), 4 warning(s) (0 minute(s), 0 second(s)) ===|

```

```
39     } printf("%s\n", line);
40 }
41
42 void function()
43 {
44     char * data;
45     char dataBuffer[100] = COMMAND_ARG2;
46     data = dataBuffer;
47     if (100 - dataLen > 1)
48     {
49         pFile = fopen(FILENAME, "r");
50         if (pFile != NULL)
51         {
52             if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
53             {
54                 printLine("fgets() failed");
55                 data[dataLen] = '\0';
56                 fclose(pFile);
57             }
58             else
59             {
60                 fclose(pFile);
61             }
62         }
63     }
64 }
65
66 {
67     char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
68     EXECV(COMMAND_INT_PATH, args); // 这里 COMMAND_ARG2 安全
69 }
70
71 int main(int argc, char * argv[])
72 {
73     /* seed randomness */
74     srand( (unsigned)time(NULL) );
75     printLine("doubao...");
76     function();
77 }
78
79
```

cpp & others

Search results × Cccc × Build log × Build messages × CppCheck/Ver++ × CppCheck/Ver++ messages × Cscope × Debugger × DoxyBlocks × Fortran info ×

File	Line	Message
		*** Build file: "no target" in "no project" (compiler: unknown) ***
D:\Software...		In function 'function':
D:\Software...	48	error: 'dataLen' undeclared (first use in this function). did you mean 'data'?
D:\Software...	48	note: each undeclared identifier is reported only once for each function it...
D:\Software...	50	error: 'pFile' undeclared (first use in this function)

(3) ChatGPT

测试输入:



测试输出:

漏洞分析

1. 缓冲区溢出:

- 这段代码涉及缓冲区 `dataBuffer`，并使用 `fgets` 读取数据到 `data`：

```
cpp
size_t dataLen = strlen(data);
if (100 - dataLen > 1)
{
    pFile = fopen(FILENAME, "r");
    if (pFile != NULL)
    {
        if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
        {
            printf("fgets() failed");
            data[dataLen] = '\0';
        }
        fclose(pFile);
    }
}
```

- `dataBuffer` 的大小是 100 字节，如果 `fgets` 从文件中读取的数据超过 100 字节时，将会导致缓冲区溢出。虽然你控制了读取长度 `100 - dataLen`，但仍需确保 `data` 指向的缓冲区大小足够。

2. 文件路径未验证:

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

2. 文件路径未验证:

- `FILENAME` 是文件路径的变量，但没有对其进行验证。如果这个路径是由外部输入提供的，可能会导致路径遍历攻击，或者读取敏感文件。因此，需要验证或过滤文件路径。

3. 命令注入风险:

- 在执行命令时，使用了 `COMMAND_INT_PATH` 和其他命令参数：

```
cpp
char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
EXECV(COMMAND_INT_PATH, args);
```

- 如果这些命令路径或参数来自不受信任的输入，可能存在命令注入的风险，攻击者可能利用这些输入执行任意命令。

修复建议

1. 确保缓冲区大小正确:

- 要保证 `fgets` 读取的数据不会导致缓冲区溢出。可以通过显式检查缓冲区的大小，并确保读取的内容不会超出其边界。

2. 验证文件路径:

- 对文件路径 `FILENAME` 进行验证，确保其合法性，防止路径遍历攻击。

3. 验证命令路径和参数:

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析:

该模型找到了测试代码的缓冲区溢出、命令注入漏洞并提供了漏洞所在行:

```
size_t dataLen = strlen(data);
if (100 - dataLen > 1)
{
    pFile = fopen(FILENAME, "r");
    if (pFile != NULL)
    {
        if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
        {
            printLine("fgets() failed");
        }
    }
}
```



```

        data[dataLen] = '\0';
    }
    fclose(pFile);
}
}

```

```

char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3,
NULL};EXECV(COMMAND_INT_PATH, args);

```

模型修改的代码：

```

void function()
{
    char dataBuffer[100] = COMMAND_ARG2;
    char * data = dataBuffer;

    if (STATIC_CONST_FIVE == 5)
    {
        size_t dataLen = strlen(data);
        FILE * pFile;
        // 确保不会发生缓冲区溢出
        if (100 - dataLen > 1)
        {
            pFile = fopen(FILENAME, "r");
            if (pFile != NULL)
            {
                if (fgets(data + dataLen, (int)(100 - dataLen), pFile) == NULL)
                {
                    printLine("fgets() failed");
                    data[dataLen] = '\0'; // 确保字符串以 \0 结束
                }
                fclose(pFile);
            }
        }
    }

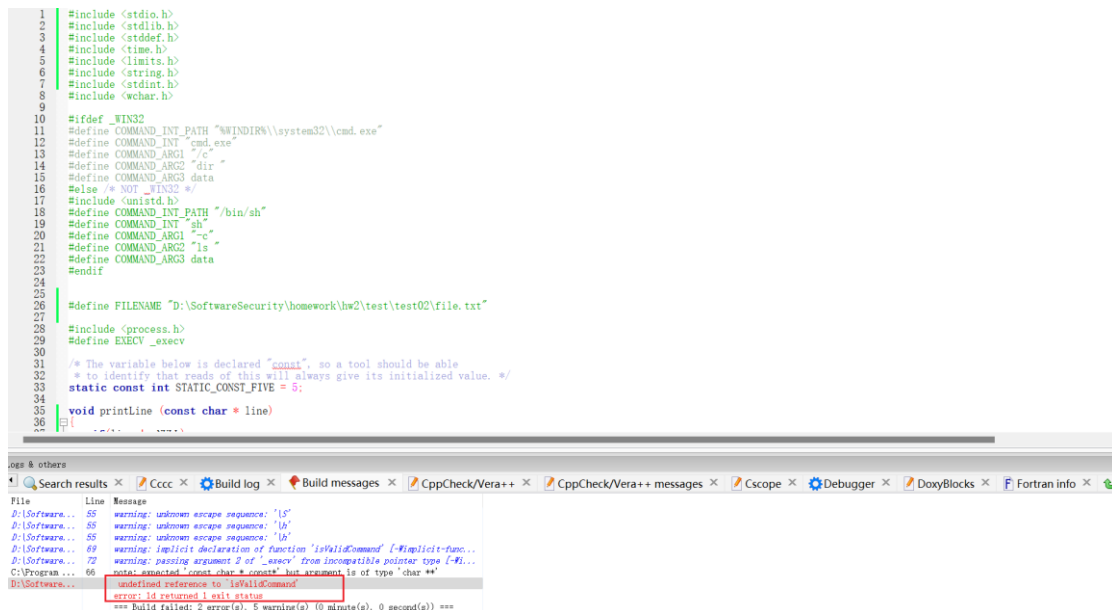
    // 验证命令路径和参数，确保不被注入
    if (isValidCommand(COMMAND_INT_PATH)) // 假设 isValidCommand 是检查命令路径的函数
    {
        char *args[] = {COMMAND_INT_PATH, COMMAND_ARG1, COMMAND_ARG3, NULL};
        EXECV(COMMAND_INT_PATH, args);
    }
}

```

代码运行结果：运行 test02/chatgpt.c:

代码运行失败:

```
||=== Build file: "no target" in "no project" (compiler: unknown) ===|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|In function 'function':|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|55|warning: unknown escape
sequence: '\S'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|55|warning: unknown escape
sequence: '\h'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|55|warning: unknown escape
sequence: '\h'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|69|warning: implicit declaration
of function 'isValidCommand' [-Wimplicit-function-declaration]|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.c|72|warning: passing argument 2 of
'_execv' from incompatible pointer type [-Wincompatible-pointer-types]|
C:\Program Files\CodeBlocks\MinGW\x86_64-w64-mingw32\include\process.h|66|note: expected
'const char * const*' but argument is of type 'char **'|
D:\SoftwareSecurity\homework\hw2\test\test02\chatgpt.o:chatgpt.c|| undefined reference
to `isValidCommand'|
||error: ld returned 1 exit status|
||=== Build failed: 2 error(s), 5 warning(s) (0 minute(s), 0 second(s)) ===|
```



```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <limits.h>
6  #include <process.h>
7  #include <unistd.h>
8  #include <wchar.h>
9
10 #ifdef _WIN32
11 #define COMMAND_INT_PATH "%WINDIR%\\system32\\cmd.exe"
12 #define COMMAND_ARG1 "/c"
13 #define COMMAND_ARG2 "dir"
14 #define COMMAND_ARG3 data
15 #else /* NOT _WIN32 */
16 #include <unistd.h>
17 #define COMMAND_INT_PATH "/bin/sh"
18 #define COMMAND_ARG1 "-c"
19 #define COMMAND_ARG2 "ls"
20 #define COMMAND_ARG3 data
21 #endif
22
23 #define FILENAME "D:\\SoftwareSecurity\\homework\\hw2\\test\\test02\\file.txt"
24
25 #include <process.h>
26 #define EXECV _execv
27
28 /* The variable below is declared "const", so a tool should be able
29 * to identify that reads of this will always give its initialized value. */
30 static const int STATIC_CONST_FIVE = 5;
31
32 void printline (const char * line)
33 {
34     printf("%s\n", line);
35 }
36
37 int function()
38 {
39     char data;
40     data = ' ';
41     printf("data: %c\n", data);
42     FILE * f = fopen(FILENAME, "r");
43     if (f == NULL)
44     {
45         printf("Error opening file: %s\n", FILENAME);
46         return 1;
47     }
48     char line[1024];
49     while (fgets(line, sizeof line, f) != NULL)
50     {
51         printline(line);
52     }
53     fclose(f);
54     return 0;
55 }
```

Build messages:

File	Line	Message
D:\Software...	55	warning: unknown escape sequence: '\S'
D:\Software...	55	warning: unknown escape sequence: '\h'
D:\Software...	55	warning: unknown escape sequence: '\h'
D:\Software...	69	warning: implicit declaration of function 'isValidCommand' [-Wimplicit-func...
D:\Software...	72	warning: passing argument 2 of '_execv' from incompatible pointer type [-Wincompa...
C:\Program...	66	note: expected 'const char * const*' but argument is of type 'char **'
D:\Software...		undefined reference to `isValidCommand'
D:\Software...		error: ld returned 1 exit status
		*** Build failed: 2 error(s), 5 warning(s) (0 minute(s), 0 second(s)) ***

3、CWE190_Integer_Overflow__char_fscanf_add_09.c

测试代码:

```
void function()
{
    char data;
    data = ' ';
```

```

if(GLOBAL_CONST_TRUE)
{
    fscanf (stdin, "%c", &data);
}
if(GLOBAL_CONST_TRUE)
{
    {
        char result = data + 1;
        printHexCharLine(result);
    }
}
}

```

运行结果：运行 test03\CWE190_Integer_Overflow__char_fscanf_add_09.c:

```

D:\SoftwareSecurity\homewo x + v
Calling good()...
1
32
0b
03
03
Finished good()
Calling bad()...
1
32
Finished bad()

Process returned 0 (0x0)   execution time : 3.883 s
Press any key to continue.

```

(1) 讯飞星火

测试输入:

阅书万卷，我是你的知识宝典

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```

void function()
{
    char data;
    data = '';
    if(GLOBAL_CONST_TRUE)
    {
        fscanf (stdin, "%c", &data);
    }
    if(GLOBAL_CONST_TRUE)
    {
        {
            char result = data + 1;
            printHexCharLine(result);
        }
    }
}

```

测试输出：

阅书万卷，我是你的知识宝典

+

🔍

📌

📁

🔄

🗑️

📄

📖

讯飞星火

在检查代码时，我注意到以下几个潜在的问题：

1. 未初始化的全局常量：GLOBAL_CONST_TRUE 未定义。如果这是一个全局常量，它应该被正确定义和初始化。

2. 输入验证缺失：fscanf 从标准输入读取字符，但没有对输入进行任何验证。这可能导致意外行为或安全漏洞。

3. 缓冲区溢出风险：虽然 data 是一个字符变量，但在某些情况下，如果输入数据超过一个字符，可能会导致缓冲区溢出。

4. 未包含必要的头文件：代码中使用了 fscanf 和 printHexCharLine，但这些函数可能来自自定义库或标准库，需要确保包含相应的头文件。

假设 GLOBAL_CONST_TRUE 是布尔值，并且 printHexCharLine 是一个有效的函数，以下是修复后的代码：

c

复制代码

```
#include <stdio.h>
#include <stdbool.h>

// 假设 printHexCharLine 是一个有效的函数声明
void printHexCharLine(char c);

void function()
{

```

代码纠错

内容写作

文本润色

网页摘要

中英翻译

学习计划

居家健身

更多

🗨️ 全新对话

点这里，尽管问，Shift+Enter换行

🗨️

📤

内容由讯飞星火大模型生成，仅供参考

阅书万卷，我是你的知识宝典

+

🔍

📌

📁

🔄

🗑️

📄

📖

讯飞星火

```
void printHexCharLine(char c);

void function()
{
    char data;
    data = ' ';
    bool GLOBAL_CONST_TRUE = true; // 确保 GLOBAL_CONST_TRUE 被正确定义和初始化

    if (GLOBAL_CONST_TRUE)
    {
        // 使用 fgets 代替 fscanf 以避免缓冲区溢出
        if (fgets(kdata, sizeof(data), stdin) != NULL)
        {
            // 仅处理第一个字符
            data = data[0];
        }
    }

    if (GLOBAL_CONST_TRUE)
    {
        char result = data + 1;
        printHexCharLine(result);
    }
}

```

代码纠错

内容写作

文本润色

网页摘要

中英翻译

学习计划

居家健身

更多

🗨️ 全新对话

点这里，尽管问，Shift+Enter换行

🗨️

📤

内容由讯飞星火大模型生成，仅供参考

24 / 113



结果分析：

该模型找到了测试代码的缓冲区溢出、命令注入漏洞，但未提供漏洞所在行。

模型修改的代码：

```
void function()
{
    char data;
    data = ' ';
    bool GLOBAL_CONST_TRUE = true; // 确保 GLOBAL_CONST_TRUE 被正确定义和初始化

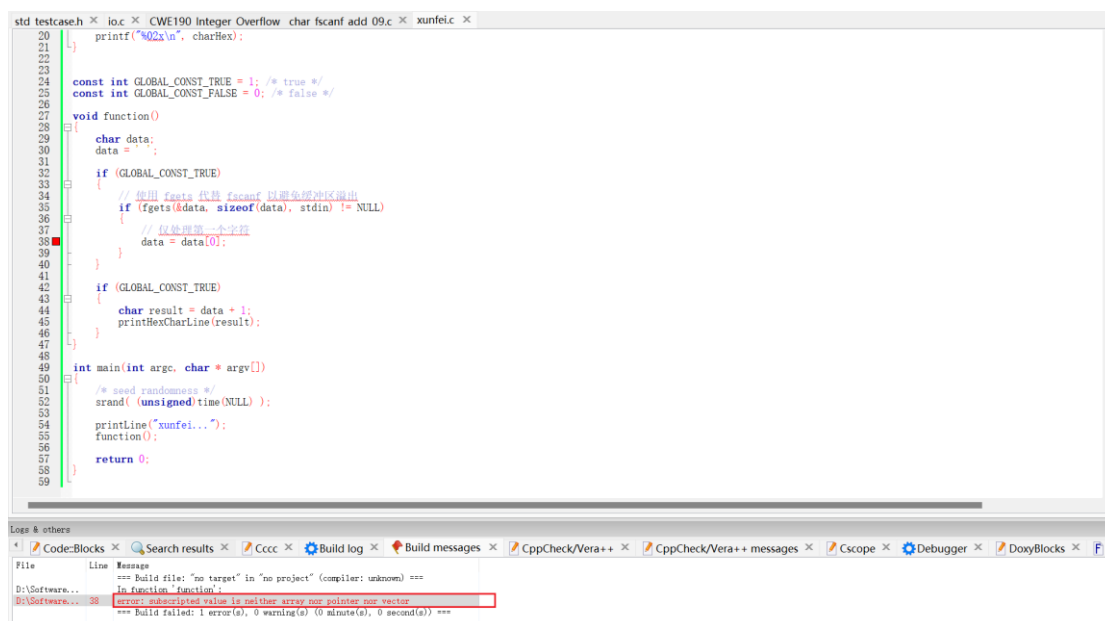
    if (GLOBAL_CONST_TRUE)
    {
        // 使用 fgets 代替 fscanf 以避免缓冲区溢出
        if (fgets(&data, sizeof(data), stdin) != NULL)
        {
            // 仅处理第一个字符
            data = data[0];
        }
    }

    if (GLOBAL_CONST_TRUE)
    {
        char result = data + 1;
        printHexCharLine(result);
    }
}
```

代码运行结果：运行 test03/xunfei.c:

出现报错，说明修改代码存在问题。

```
||=== Build file: "no target" in "no project" (compiler: unknown) ===|
D:\SoftwareSecurity\homework\hw2\test\test03\xunfei.c||In function 'function':|
D:\SoftwareSecurity\homework\hw2\test\test03\xunfei.c|38|error: subscripted value is
neither array nor pointer nor vector|
||=== Build failed: 1 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===|
```

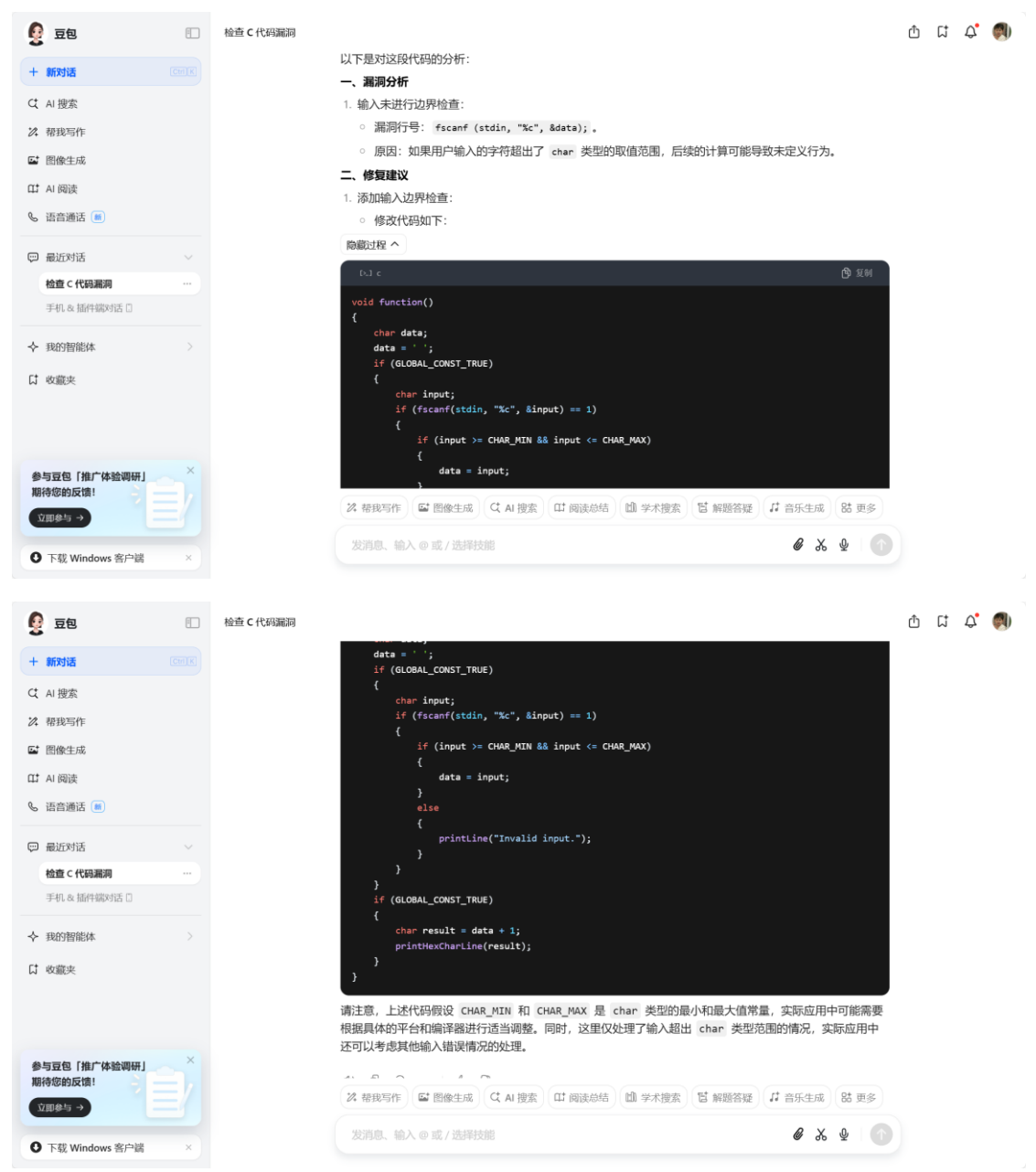


(2) 豆包

测试输入:



测试输出：



结果分析：

该模型找到了测试代码的输入未进行边界检查漏洞并提供了漏洞所在行：

```
fscanf (stdin, "%c", &data);
```

模型修改的代码：

```
void function()
{
    char data;
```

```

data = ' ';
if (GLOBAL_CONST_TRUE)
{
    char input;
    if (fscanf(stdin, "%c", &input) == 1)
    {
        if (input >= CHAR_MIN && input <= CHAR_MAX)
        {
            data = input;
        }
        else
        {
            printLine("Invalid input.");
        }
    }
}
if (GLOBAL_CONST_TRUE)
{
    char result = data + 1;
    printHexCharLine(result);
}
}

```

代码运行结果：运行 test03/doubao.c:

代码可以正常运行，但输出结果与 bad()一致。

```

D:\SoftwareSecurity\homewo x + v
doubao...
1
32
Process returned 0 (0x0) execution time : 1.390 s
Press any key to continue.

```

(3) ChatGPT

测试输入：

ChatGPT

共享

DA

请你检查以下代码是否存在漏洞。如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    char data;
    data = ' ';
    if(GLOBAL_CONST_TRUE)
    {
        fscanf (stdin, "%c", &data);
    }
    if(GLOBAL_CONST_TRUE)
    {
        {
            char result = data + 1;
            printHexCharLine(result);
        }
    }
}
```

测试输出：

ChatGPT

共享

DA

漏洞分析

1. 格式化输入不受控:

- 在这段代码中，`fscanf` 读取用户输入的字符并将其存储在 `data` 变量中:

```
cpp
fscanf(stdin, "%c", &data);
```

- `fscanf` 使用 `%c` 格式化符号读取单个字符。然而，`stdin` 是一个来自用户输入的流，这意味着输入的数据并未经过有效的验证。用户可能输入一个非法字符（例如控制字符），导致后续代码可能不按预期工作。

2. 未初始化的 `data` 变量:

- 虽然代码给 `data` 赋了一个初值 `' '`（空格），但这个赋值仅在输入部分条件成立时才会被覆盖。如果程序在未读取输入的情况下执行 `fscanf`，`data` 可能含有无效或意外的值。

3. 潜在的未定义行为:

- 在以下行:

```
cpp
char result = data + 1;
```

`data` 被增加了 1，这在大多数情况下是有效的，但如果 `data` 是一个极端的字符值（例如控制字符或值大于空字符类型的最大值）可能会导致未定义行为。由于 `data` 是从 `stdin`

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请检查重要信息。

29 / 113

- 11.11.1.1.1.1.

```
cpp
char result = data + 1;
```

复制代码

`data` 被增加了 1，这在大多数情况下是有效的，但如果 `data` 是一个极端的字符值（例如控制字符或值大于字符类型的最大值），可能会导致未定义行为。由于 `data` 是从 `stdin` 输入的，攻击者可以尝试输入不期望的字符（例如，负数或超过 `char` 类型最大值的数值）。

4. 没有处理输入错误:

- 没有对 `fscanf` 的返回值进行检查，这可能导致潜在的输入错误未被检测到。如果输入格式不正确，`fscanf` 会失败并不会按预期行为继续执行。

修复建议

1. 确保输入数据是有效的:

- 应该验证 `fscanf` 的返回值，以确保输入操作成功并且输入是有效的。

2. 处理潜在的字符溢出或异常值:

- 确保 `data` 在执行任何算术操作之前是有效的，并且不会溢出 `char` 类型的范围。

3. 检查输入的边界:

- 如果输入不符合预期，可以提供反馈并要求重新输入。

修复后的代码:

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

?

修复后的代码:

```
cpp
#include <stdio.h>

void function()
{
    char data = ' '; // 初始化为一个有效字符
    if (GLOBAL_CONST_TRUE)
    {
        if (fscanf(stdin, "%c", &data) != 1) // 确保 fscanf 成功
        {
            printf("Input error or invalid format");
            return; // 如果输入错误，则返回
        }
    }

    if (GLOBAL_CONST_TRUE)
    {
        // 如果 data 是一个有效字符，且不溢出，进行加法操作
        if (data >= 0 && data < 255) // 确保数据有效
        {
            char result = data + 1;
            printfHexCharLine(result);
        }
    }
}
```

复制代码

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

?



结果分析:

该模型找到了测试代码的格式化输入不受控漏洞并提供了漏洞所在行:

```
fscanf(stdin, "%c", &data);
```

模型修改的代码:

```
void function()
{
    char data = ' '; // 初始化为一个有效字符
    if (GLOBAL_CONST_TRUE)
    {
        if (fscanf(stdin, "%c", &data) != 1) // 确保 fscanf 成功
        {
            printLine("Input error or invalid format");
            return; // 如果输入错误，则返回
        }
    }

    if (GLOBAL_CONST_TRUE)
    {
        // 如果 data 是一个有效字符，且不溢出，进行加法操作
        if (data >= 0 && data < 255) // 确保数据有效
        {
            char result = data + 1;
            printHexCharLine(result);
        }
        else
    }
```

```

    {
        printLine("Invalid character value");
    }
}
}

```

代码运行结果：运行 test03/chatgpt.c:

代码可以正常运行，但输出结果与 bad()一致。

```

chatgpt...
1
32

Process returned 0 (0x0)   execution time : 2.291 s
Press any key to continue.

```

4、CWE78_OS_Command_Injection__wchar_t_environment_system_03.c

测试代码：

```

void function()
{
    wchar_t * data;
    wchar_t data_buf[100] = FULL_COMMAND;
    data = data_buf;
    if(5==5)
    {
        {
            size_t dataLen = wcslen(data);
            wchar_t * environment = GETENV(ENV_VARIABLE);
            if (environment != NULL)
            {
                wcsncat(data+dataLen, environment, 100-dataLen-1);
            }
        }
    }
}

```

```

    }
    if (SYSTEM(data) != 0)
    {
        printLine("command execution failed!");
        exit(1);
    }
}

```

运行结果：运行

test04\CWE78_OS_Command_Injection__wchar_t_environment_system_03.c:

Calling good()...

驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录

日期/时间	名称	大小
2024/11/12 11:28	<DIR>	.
2024/11/08 15:53	<DIR>	..
2024/11/12 11:27	chatgpt.c	1,346
2024/11/12 11:27	chatgpt.exe	55,791
2024/11/12 11:27	chatgpt.o	1,831
2024/11/12 11:23	CWE78_OS_Command_Injection__wchar_t_environment_system_03.c	3,193
2024/11/12 11:28	CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe	57,582
2024/11/12 11:28	CWE78_OS_Command_Injection__wchar_t_environment_system_03.o	2,879
2024/11/12 11:26	doubao.c	1,600
2024/11/12 11:28	doubao.exe	56,656
2024/11/12 11:28	doubao.o	1,950
2024/11/05 23:01	test04.txt	2,968
2024/11/12 11:23	xunfei.c	1,518
2024/11/12 11:24	xunfei.exe	57,164
2024/11/12 11:24	xunfei.o	2,040
13 个文件		246,518 字节
2 个目录		98,941,857,792 可用字节

驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录

日期/时间	名称	大小
2024/11/12 11:28	<DIR>	.
2024/11/08 15:53	<DIR>	..
2024/11/12 11:27	chatgpt.c	1,346
2024/11/12 11:27	chatgpt.exe	55,791
2024/11/12 11:27	chatgpt.o	1,831
2024/11/12 11:23	CWE78_OS_Command_Injection__wchar_t_environment_system_03.c	3,193
2024/11/12 11:28	CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe	57,582
2024/11/12 11:28	CWE78_OS_Command_Injection__wchar_t_environment_system_03.o	2,879
2024/11/12 11:26	doubao.c	1,600
2024/11/12 11:28	doubao.exe	56,656
2024/11/12 11:28	doubao.o	1,950
2024/11/05 23:01	test04.txt	2,968
2024/11/12 11:23	xunfei.c	1,518
2024/11/12 11:24	xunfei.exe	57,164
2024/11/12 11:24	xunfei.o	2,040

```
D:\SoftwareSecurity\homewo x + v
2024/11/12 11:27 1,346 chatgpt.c
2024/11/12 11:27 55,791 chatgpt.exe
2024/11/12 11:27 1,831 chatgpt.o
2024/11/12 11:23 3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:28 57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:28 2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26 1,600 doubao.c
2024/11/12 11:28 56,656 doubao.exe
2024/11/12 11:28 1,950 doubao.o
2024/11/05 23:01 2,968 test04.txt
2024/11/12 11:23 1,518 xunfei.c
2024/11/12 11:24 57,164 xunfei.exe
2024/11/12 11:24 2,040 xunfei.o
13 个文件 246,518 字节
2 个目录 98,941,857,792 可用字节
Finished good()
Calling bad()...
驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录
2024/11/12 11:28 <DIR> .
2024/11/08 15:53 <DIR> ..
2024/11/12 11:27 1,346 chatgpt.c
2024/11/12 11:27 55,791 chatgpt.exe
2024/11/12 11:27 1,831 chatgpt.o
2024/11/12 11:23 3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:28 57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:28 2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26 1,600 doubao.c
2024/11/12 11:28 56,656 doubao.exe
2024/11/12 11:28 1,950 doubao.o
2024/11/05 23:01 2,968 test04.txt
2024/11/12 11:23 1,518 xunfei.c
2024/11/12 11:24 57,164 xunfei.exe
2024/11/12 11:24 2,040 xunfei.o
13 个文件 246,518 字节
2 个目录 98,941,857,792 可用字节
Finished bad()
Process returned 0 (0x0) execution time : 0.205 s
Press any key to continue.
```

(1) 讯飞星火

测试输入：



测试输出：

阅书万卷，我是你的知识宝典

超拟人数字人

新建群聊

新建智能体

讯飞星火

这段代码存在多个潜在的漏洞和问题。以下是详细的分析和修复建议：

- 缓冲区溢出**：`wcsncat(data+dataLen, environment, 100-dataLen-1)`：这一行代码没有检查 `environment` 的长度，可能会导致缓冲区溢出。
- 未定义的宏**：`FULL_COMMAND`，`GETENV`，`ENV_VARIABLE`，`SYSTEM`，和 `printLine` 这些宏或函数在代码中没有定义，需要确保它们在其他地方正确定义。
- 硬编码长度**：`100` 这个长度是硬编码的，应该使用常量或宏来代替，以提高代码的可维护性。
- 不安全的系统调用**：直接使用 `SYSTEM` 函数执行命令可能存在安全风险，应考虑使用更安全的方法。

下面是修复后的代码：

c

复制代码

```
#include <stdlib.h>
#include <wchar.h>
#include <stdio.h>

// 假设这些宏和函数在其他地方已经定义
#define FULL_COMMAND L"initial command "
#define ENV_VARIABLE "MY_ENV_VAR"
#define MAX_DATA_LEN 100
```

代码纠错

内容写作

文本润色

网页摘要

中英翻译

学习计划

居家健身

儿童教育

更多

全新对话

点这里，尽管问，Shift+Enter换行

内容由讯飞星火大模型生成，仅供参考

阅书万卷，我是你的知识宝典

超拟人数字人

新建群聊

新建智能体

修复后的代码

```
void printLine(const wchar_t * message) {
    wprintf(L"%s\n", message);
}

int main() {
    wchar_t data[MAX_DATA_LEN] = FULL_COMMAND;
    size_t dataLen = wcslen(data);
    wchar_t * environment = getenv(ENV_VARIABLE); // 假设 getenv 返回的是宽字符串

    if (environment != NULL) {
        size_t envLen = wcslen(environment);
        if (dataLen + envLen < MAX_DATA_LEN - 1) {
            wcsncat(data, environment, MAX_DATA_LEN - dataLen - 1);
        } else {
            printLine(L"Environment variable is too long!");
            exit(1);
        }
    }

    if (system(data) != 0) { // 假设 system 接受宽字符串
        printLine(L"command execution failed!");
        exit(1);
    }
}
```

代码纠错

内容写作

文本润色

网页摘要

中英翻译

学习计划

居家健身

儿童教育

更多

全新对话

点这里，尽管问，Shift+Enter换行

内容由讯飞星火大模型生成，仅供参考



结果分析：

该模型找到了测试代码的缓冲区溢出、不安全的系统调用漏洞并提供了漏洞所在行：

```
wcsncat(data+dataLen, environment, 100-dataLen-1);
```

模型修改的代码：

```
#include <stdlib.h>
#include <wchar.h>
#include <stdio.h>

// 假设这些宏和函数在其他地方已经定义
#define FULL_COMMAND L"initial command "
#define ENV_VARIABLE "MY_ENV_VAR"
#define MAX_DATA_LEN 100

void printLine(const wchar_t * message) {
    wprintf(L"%ls\n", message);
}

int main() {
    wchar_t data[MAX_DATA_LEN] = FULL_COMMAND;
    size_t dataLen = wcslen(data);
    wchar_t * environment = getenv(ENV_VARIABLE); // 假设 getenv 返回的是宽字符串

    if (environment != NULL) {
        size_t envLen = wcslen(environment);
        if (dataLen + envLen < MAX_DATA_LEN - 1) {
```



```

        wcsncat(data, environment, MAX_DATA_LEN - dataLen - 1);
    } else {
        printLine(L"Environment variable is too long!");
        exit(1);
    }
}

if (system(data) != 0) { // 假设 system 接受宽字符串
    printLine(L"command execution failed!");
    exit(1);
}

return 0;
}

```

代码运行结果：运行 test04/xunfei.c:

运行结果错误。

The screenshot shows a Windows command prompt window with the title bar 'D:\SoftwareSecurity\homewo'. The command 'xunfei...' has been entered, resulting in an error message: "'i' 不是内部或外部命令，也不是可运行的程序或批处理文件。' (The command 'i' is not an internal or external command, nor a batch file or executable program). Below the error message, it says 'Process returned 1 (0x1) execution time : 0.131 s' and 'Press any key to continue.'.

(2) 豆包

测试输入:



测试输出：





结果分析：

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行：

```
wcsncat(data + dataLen, environment, 100 - dataLen - 1);
```

模型修改的代码：

```
void function()
{
    wchar_t * data;
    wchar_t data_buf[100] = FULL_COMMAND;
    data = data_buf;
    if (5 == 5)
    {
        {
            size_t dataLen = wcslen(data);
            wchar_t * environment = GETENV(ENV_VARIABLE);
            if (environment != NULL)
            {
                size_t envLen = wcslen(environment);
                size_t remainingSpace = 100 - dataLen - 1;
                if (envLen < remainingSpace)
                {
                    wcsncat(data + dataLen, environment, envLen);
                }
            }
            else
            {
                wcsncat(data + dataLen, environment, remainingSpace);
            }
        }
    }
}
```

```

    }
}
}
}
if (SYSTEM(data) != 0)
{
    printLine("command execution failed!");
    exit(1);
}
}

```

代码运行结果：运行 test04/doubao.c:

运行结果正确。

```

doubao...
驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录
2024/11/12 11:28 <DIR>      .
2024/11/08 15:53 <DIR>      ..
2024/11/12 11:27          1,346 chatgpt.c
2024/11/12 11:27       55,791 chatgpt.exe
2024/11/12 11:27          1,831 chatgpt.o
2024/11/12 11:23          3,193 CWE78_OS_Command_Injection_wchar_t_environment_system_03.c
2024/11/12 11:23       57,582 CWE78_OS_Command_Injection_wchar_t_environment_system_03.exe
2024/11/12 11:23       2,879 CWE78_OS_Command_Injection_wchar_t_environment_system_03.o
2024/11/12 11:26          1,600 doubao.c
2024/11/12 11:28       56,656 doubao.exe
2024/11/12 11:28          1,950 doubao.o
2024/11/05 23:01          2,968 test04.txt
2024/11/12 11:23          1,518 xunfei.c
2024/11/12 11:24          57,164 xunfei.exe
2024/11/12 11:24          2,040 xunfei.o
                13 个文件          246,518 字节
                2 个目录 98,941,857,792 可用字节

Process returned 0 (0x0)   execution time : 0.150 s
Press any key to continue.
|

```

(3) ChatGPT

测试输入:

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    wchar_t * data;
    wchar_t data_buf[100] = FULL_COMMAND;
    data = data_buf;
    if ($==5)
    {
        {
            size_t dataLen = wcslen(data);
            wchar_t * environment = GETENV(ENV_VARIABLE);
            if (environment != NULL)
            {
                wcsncat(data+dataLen, environment, 100-dataLen-1);
            }
        }
    }
    if (SYSTEM(data) != 0)
    {
        printLine("command execution failed!");
        exit(1);
    }
}
```

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请检查重要信息。

测试输出：

此代码存在安全漏洞，特别是命令注入漏洞。以下是漏洞行及原因分析：

漏洞分析

- 第10行: `wcsncat(data+dataLen, environment, 100-dataLen-1);`
 - 原因: 从环境变量获取的内容直接附加到命令字符串 `data` 上，而没有对内容进行任何验证或过滤。环境变量可能包含恶意内容，使得整个字符串 `data` 中的命令可以被攻击者操控。这将导致潜在的命令注入风险。
- 第14行: `SYSTEM(data);`
 - 原因: `SYSTEM` 函数直接执行由 `data` 指定的命令字符串。因此，如果 `data` 被恶意修改，攻击者可以通过该函数执行任意命令。

修复建议

- 使用白名单或过滤机制验证环境变量的内容，防止其包含不受信任的字符或命令。
- 如果可行，避免使用 `SYSTEM` 函数。可以考虑使用更安全的 API，如 `CreateProcess` (Windows) 或 `execve` (Linux) 等。
- 只允许预定义的命令执行，避免将用户输入直接拼接到命令中。

修复后的代码

以下是修复后的代码，通过移除环境变量的拼接来避免命令注入问题：

```
c
```

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请检查重要信息。



结果分析:

该模型找到了测试代码的命令注入漏洞并提供了漏洞所在行:

```
wcsncat(data+dataLen, environment, 100-dataLen-1);
SYSTEM(data);
```

模型修改的代码:

```
void function()
{
    wchar_t * data;
    wchar_t data_buf[100] = FULL_COMMAND;
```

```

data = data_buf;
if(5==5)
{
    // 移除对环境变量的读取
    printLine("Environment variable usage removed for security.");
}
// 限制 data 的内容为安全命令
if (wcscmp(data, FULL_COMMAND) == 0) // 假设 FULL_COMMAND 是一个安全的命令
{
    if (SYSTEM(data) != 0)
    {
        printLine("command execution failed!");
        exit(1);
    }
}
else
{
    printLine("Untrusted command, execution aborted.");
}
}

```

代码运行结果：运行 test04/chatgpt.c:

运行结果正确。

```

chatgpt...
Environment variable usage removed for security.
驱动器 D 中的卷是 Data
卷的序列号是 6E99-8D8D

D:\SoftwareSecurity\homework\hw2\test\test04 的目录
2024/11/12 11:27 <DIR>      .
2024/11/08 15:53 <DIR>      ..
2024/11/12 11:27          1,346 chatgpt.c
2024/11/12 11:27        55,791 chatgpt.exe
2024/11/12 11:27          1,831 chatgpt.o
2024/11/12 11:23      3,193 CWE78_OS_Command_Injection__wchar_t_environment_system_03.c
2024/11/12 11:23    57,582 CWE78_OS_Command_Injection__wchar_t_environment_system_03.exe
2024/11/12 11:23      2,879 CWE78_OS_Command_Injection__wchar_t_environment_system_03.o
2024/11/12 11:26          1,600 doubao.c
2024/11/12 11:26     56,656 doubao.exe
2024/11/12 11:26          1,950 doubao.o
2024/11/05 23:01          2,968 test04.txt
2024/11/12 11:23          1,518 xunfei.c
2024/11/12 11:24      57,164 xunfei.exe
2024/11/12 11:24          2,040 xunfei.o
          13 个文件          246,518 字节
           2 个目录 98,941,857,792 可用字节

Process returned 0 (0x0)   execution time : 0.148 s
Press any key to continue.

```

5、CWE244_Heap_Inspection__w32_char_free_17.c

测试代码：

```

void function()
{
    int j;
    for(j = 0; j < 1; j++)
    {
        {
            char * password = (char *)malloc(100*sizeof(char));
            if (password == NULL) {exit(-1);}
            size_t passwordLen = 0;
            HANDLE hUser;
            char * username = "User";
            char * domain = "Domain";
            password[0] = '\\0';
            if (fgets(password, 100, stdin) == NULL)
            {
                printLine("fgets() failed");
                password[0] = '\\0';
            }
            passwordLen = strlen(password);
            if (passwordLen > 0)
            {
                password[passwordLen-1] = '\\0';
            }
            if (LogonUserA(
                username,
                domain,
                password,
                LOGON32_LOGON_NETWORK,
                LOGON32_PROVIDER_DEFAULT,
                &hUser) != 0)
            {
                printLine("User logged in successfully.");
                CloseHandle(hUser);
            }
            else
            {
                printLine("Unable to login.");
            }
            free(password);
        }
    }
}

```

运行结果：运行 test05\CWE244_Heap_Inspection__w32_char_free_17.c:


```
D:\SoftwareSecurity\homewo x + v - □ x
Calling good()...
bupt
Unable to login.
Finished good()
Calling bad()...
bupt
Unable to login.
Finished bad()

Process returned 0 (0x0)   execution time : 7.027 s
Press any key to continue.
```

(1) 讯飞星火

测试输入：

阅书万卷，我是你的知识宝典

超拟人数字人 新建群聊 新建智能体

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    int j;
    for(j = 0; j < 1; j++)
    {
        char * password = (char *)malloc(100*sizeof(char));
        if (password == NULL) {exit(-1);}
        size_t passwordLen = 0;
        HANDLE hUser;
        char * username = "User";
        char * domain = "Domain";
        password[0] = '\0';
        if (fgets(password, 100, stdin) == NULL)
        {
            printLine("fgets() failed");
            password[0] = '\0';
        }
        passwordLen = strlen(password);
        if (passwordLen > 0)
        {
            password[passwordLen-1] = '\0';
        }
    }
}
```

代码纠错 内容写作 文本润色 网页摘要 中英翻译 学习计划 居家健身 儿童教育 更多 全新对话

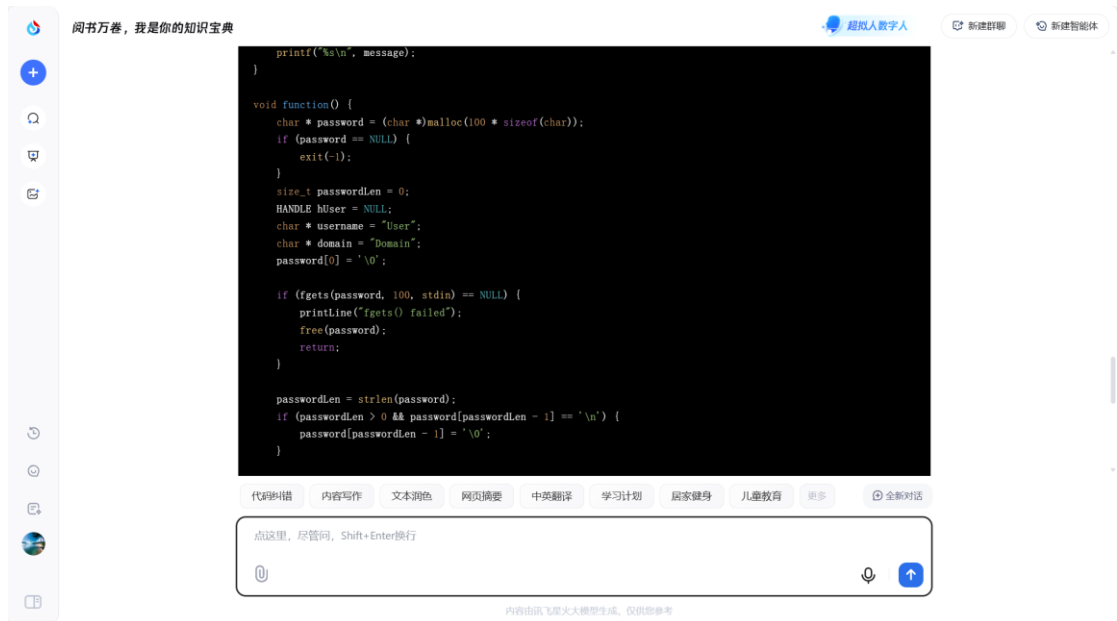
点这里，尽管问，Shift+Enter换行

内容由讯飞星火大模型生成，仅供参考



测试输出：





结果分析：

该模型找到了测试代码的内存泄漏、缓冲区溢出等漏洞但并未提供漏洞所在行。

模型修改的代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>

void printLine(const char * message) {
    printf("%s\n", message);
}
```

```

}

void function() {
    char * password = (char *)malloc(100 * sizeof(char));
    if (password == NULL) {
        exit(-1);
    }
    size_t passwordLen = 0;
    HANDLE hUser = NULL;
    char * username = "User";
    char * domain = "Domain";
    password[0] = '\\0';

    if (fgets(password, 100, stdin) == NULL) {
        printLine("fgets() failed");
        free(password);
        return;
    }

    passwordLen = strlen(password);
    if (passwordLen > 0 && password[passwordLen - 1] == '\\n') {
        password[passwordLen - 1] = '\\0';
    }

    if (LogonUserA(username, domain, password, LOGON32_LOGON_NETWORK,
LOGON32_PROVIDER_DEFAULT, &hUser)) {
        printLine("User logged in successfully.");
        CloseHandle(hUser);
    } else {
        printLine("Unable to login.");
    }

    free(password);
}

```

代码运行结果：运行 test05/xunfei.c:

运行结果正确。

```
D:\SoftwareSecurity\homewo x + v - □ x
xunfei...
bupt
Unable to login.

Process returned 0 (0x0)   execution time : 5.293 s
Press any key to continue.
```

(2) 豆包

测试输入:

豆包

+ 新对话

AI 搜索

帮我写作

图像生成

AI 阅读

语音通话

最近对话

检查 C 代码漏洞

在线教育平台定期备案

手机 & 插件锁对话

我的智能体

收藏夹

参与豆包「推广体验调研」
期待您的反馈!

立即参与

下载 Windows 客户端

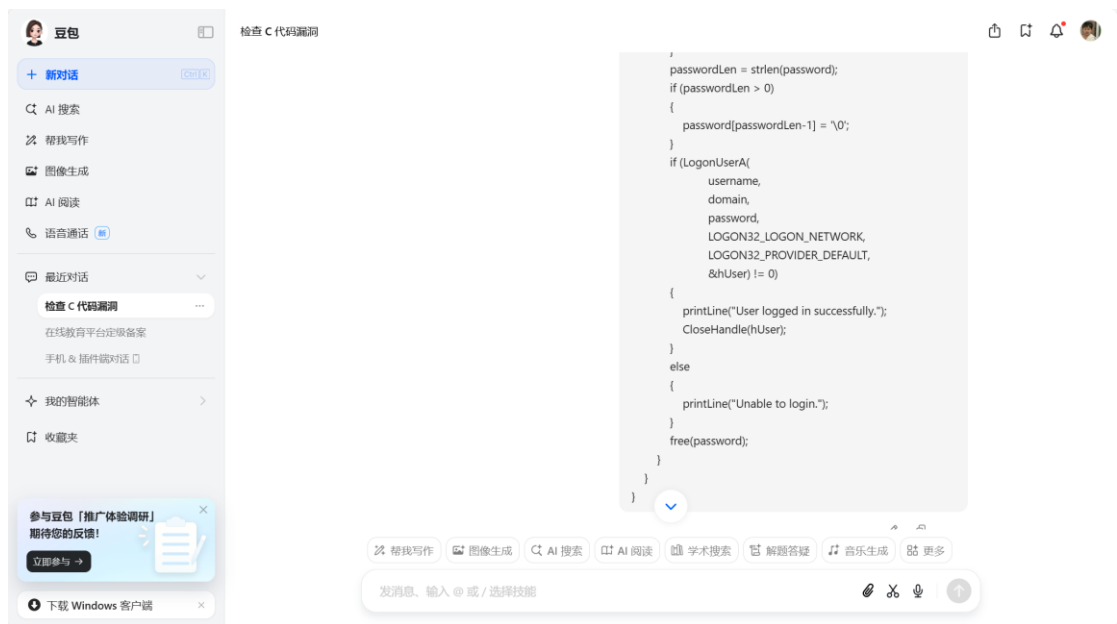
检查 C 代码漏洞

请你检查以下代码是否存在漏洞。如果存在请标出漏洞行号和原因，并给出修复后的代码

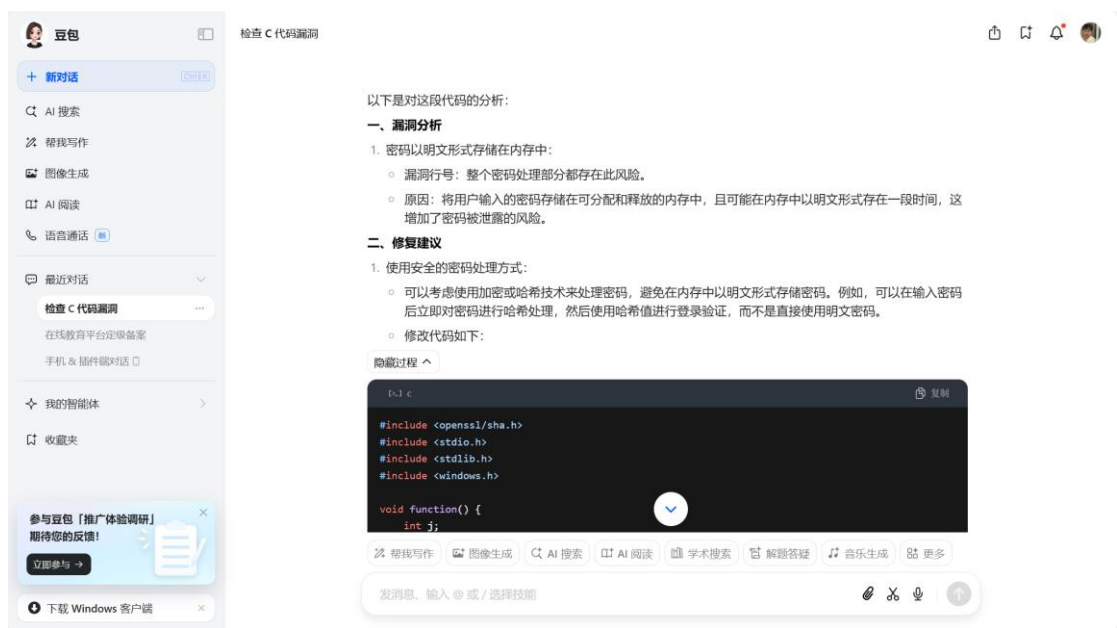
void function()
{
 int j;
 for(j = 0; j < 1; j++)
 {
 char * password = (char *)malloc(100*sizeof(char));
 if (password == NULL) {exit(-1);}
 size_t passwordLen = 0;
 HANDLE hUser;
 char * username = "User";
 char * domain = "Domain";
 password[0] = '\0';
 if (fgets(password, 100, stdin) == NULL)
 {
 printf("fgets() failed");
 password[0] = '\0';
 }
 passwordLen = strlen(password);
 if (passwordLen > 0)
 password[passwordLen-1] = '\0';
 }
}

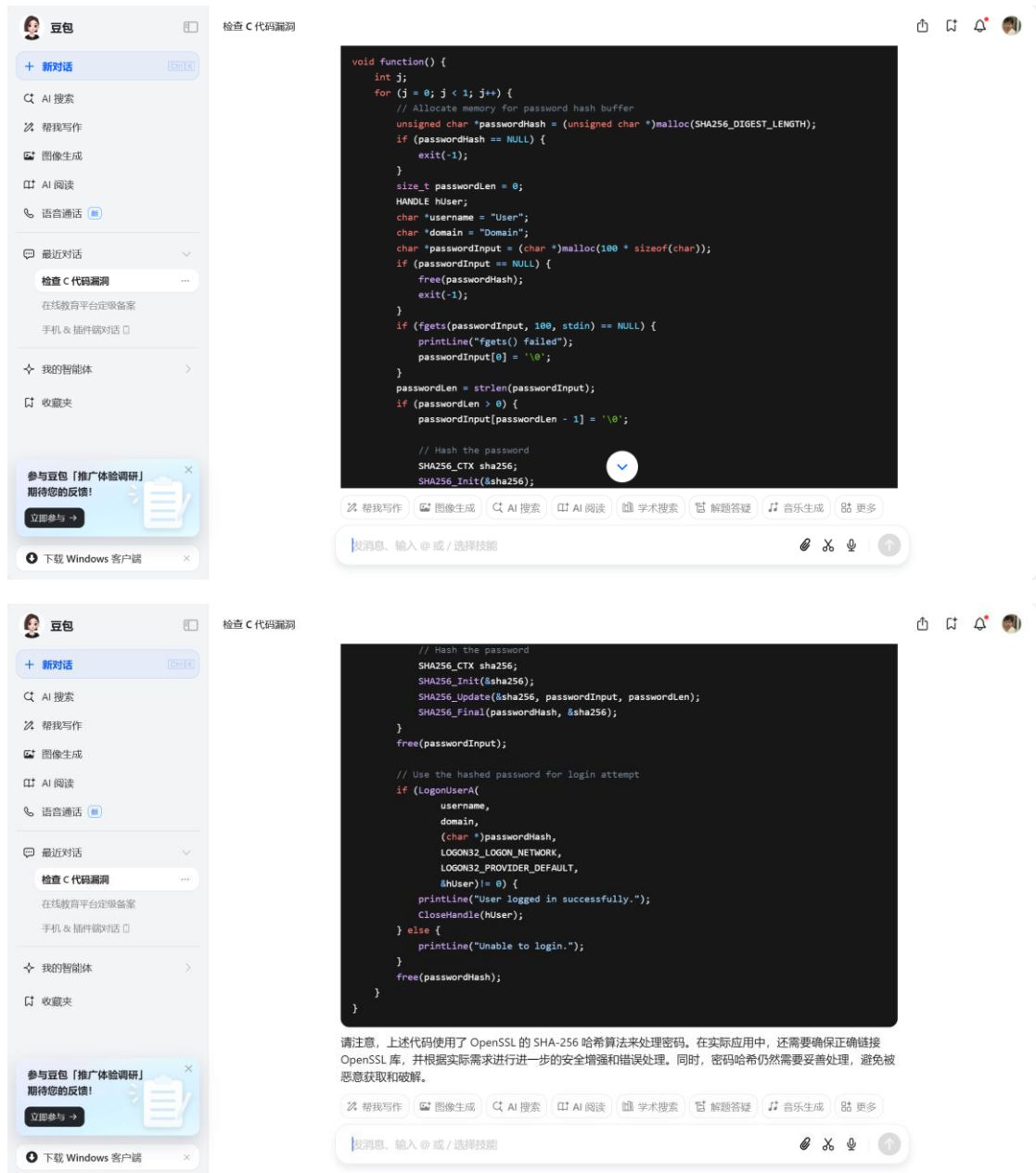
帮我写作 图像生成 AI 搜索 AI 阅读 学术搜索 解题答疑 音乐生成 更多

发消息、输入 @ 或 / 选择技能



测试输出：





结果分析：

该模型找到了测试代码的密码以明文形式存储在内存中漏洞并提供了漏洞所在行：

整个函数。

模型修改的代码：

```
#include <openssl/sha.h>
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
```

```

void function() {
    int j;
    for (j = 0; j < 1; j++) {
        // Allocate memory for password hash buffer
        unsigned char *passwordHash = (unsigned char *)malloc(SHA256_DIGEST_LENGTH);
        if (passwordHash == NULL) {
            exit(-1);
        }
        size_t passwordLen = 0;
        HANDLE hUser;
        char *username = "User";
        char *domain = "Domain";
        char *passwordInput = (char *)malloc(100 * sizeof(char));
        if (passwordInput == NULL) {
            free(passwordHash);
            exit(-1);
        }
        if (fgets(passwordInput, 100, stdin) == NULL) {
            printLine("fgets() failed");
            passwordInput[0] = '\0';
        }
        passwordLen = strlen(passwordInput);
        if (passwordLen > 0) {
            passwordInput[passwordLen - 1] = '\0';

            // Hash the password
            SHA256_CTX sha256;
            SHA256_Init(&sha256);
            SHA256_Update(&sha256, passwordInput, passwordLen);
            SHA256_Final(passwordHash, &sha256);
        }
        free(passwordInput);

        // Use the hashed password for login attempt
        if (LogonUserA(
            username,
            domain,
            (char *)passwordHash,
            LOGON32_LOGON_NETWORK,
            LOGON32_PROVIDER_DEFAULT,
            &hUser) != 0) {
            printLine("User logged in successfully.");
            CloseHandle(hUser);
        } else {

```



```

        printLine("Unable to login.");
    }
    free(passwordHash);
}
}

```

代码运行结果：运行 test05/doubao.c:

运行结果正确。

```

doubao...
bupt
Unable to login.

Process returned 0 (0x0)   execution time : 2.776 s
Press any key to continue.

```

(3) ChatGPT

测试输入:



```
printf("get password\n");
password[0] = '\0';
}
passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0';
}
if (LogonUserA(
    username,
    domain,
    password,
    LOGON32_LOGON_NETWORK,
    LOGON32_PROVIDER_DEFAULT,
    &hUser) != 0)
{
    printf("User logged in successfully.");
    CloseHandle(hUser);
}
else
{
    printf("Unable to login.");
    free(password);
}
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

?

测试输出：

此代码存在以下安全漏洞，特别是敏感数据泄露风险和内存管理问题。下面是漏洞分析及修复方法。

漏洞分析

- 第9行: `char * password = (char *)malloc(100*sizeof(char));`
 - 原因: 分配的 `password` 缓冲区用于存储敏感信息(如密码)。但是,在密码验证后,内存只是被 `free(password);` 释放,而密码数据依然可能残留在内存中,造成敏感数据泄露的风险。
- 第16行: `fgets(password, 100, stdin);`
 - 原因: `fgets` 从标准输入读取密码,但没有确保密码的长度不超过缓冲区大小。在实际使用中,如果输入过长,可能会被截断,或者潜在地导致缓冲区溢出,尽管这里限制了 100 字节,但最好显式保护读取的字节数。
- 缺少密码清除: 在 `free(password);` 前没有对密码进行安全清除,导致密码在内存释放前依然存在。

修复建议

- 清除密码缓冲区: 在 `free(password);` 之前使用 `memset` 函数清除密码,以确保密码不会残留在内存中。
- 改进输入读取: 使用安全的输入方法,如检查输入长度,避免密码被截断或溢出。
- 避免明文存储敏感数据: 尽量避免在内存中以明文形式存储敏感信息。如果可以,使用更加安全的密码处理方式。

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

?

修复后的代码

以下是改进后的代码：

```
c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void function()
{
    int j;
    for(j = 0; j < 1; j++)
    {
        char * password = (char *)malloc(100*sizeof(char));
        if (password == NULL) {exit(-1);}
        size_t passwordLen = 0;
        HANDLE hUser;
        char * username = "User";
        char * domain = "Domain";
        password[0] = '\0';

        // 限制输入长度，防止溢出
        if (fgets(password, 100, stdin) == NULL)
        {
            return;
        }
    }
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

```
// 限制输入长度，防止溢出
if (fgets(password, 100, stdin) == NULL)
{
    printf("fgets() failed");
    password[0] = '\0';
}

passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0'; // 去除末尾换行符
}

// 尝试登录
if (LogonUserA(
    username,
    domain,
    password,
    LOGON32_LOGON_NETWORK,
    LOGON32_PROVIDER_DEFAULT,
    &hUser) != 0)
{
    printf("User logged in successfully.");
    CloseHandle(hUser);
}
else
{
    return;
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析：

该模型找到了测试代码的敏感数据泄露风险和内存管理问题并提供了漏洞所在行：

```
char * password = (char *)malloc(100*sizeof(char));  
fgets(password, 100, stdin);
```

模型修改的代码：

```
#include <windows.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
void function()  
{  
    int j;  
    for(j = 0; j < 1; j++)  
    {  
        {  
            char * password = (char *)malloc(100*sizeof(char));  
            if (password == NULL) {exit(-1);}  
            size_t passwordLen = 0;  
            HANDLE hUser;  
            char * username = "User";  
            char * domain = "Domain";  
            password[0] = '\\0';
```

```

// 限制输入长度, 防止溢出
if (fgets(password, 100, stdin) == NULL)
{
    printLine("fgets() failed");
    password[0] = '\0';
}

passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0'; // 去除末尾换行符
}

// 尝试登录
if (LogonUserA(
    username,
    domain,
    password,
    LOGON32_LOGON_NETWORK,
    LOGON32_PROVIDER_DEFAULT,
    &hUser) != 0)
{
    printLine("User logged in successfully.");
    CloseHandle(hUser);
}
else
{
    printLine("Unable to login.");
}

// 在释放前清除密码缓冲区
memset(password, 0, passwordLen);
free(password);
}
}
}

```

代码运行结果：运行 test05/chatgpt.c：

运行结果正确。

```
D:\SoftwareSecurity\homewo x + v
chatgpt...
bupt
Unable to login.

Process returned 0 (0x0)   execution time : 3.386 s
Press any key to continue.
```

6、CWE253_Incorrect_Check_of_Function_Return_Value__char_fgets_13.c

测试代码：

```
void function()
{
    if(GLOBAL_CONST_FIVE==5)
    {
        {
            char dataBuffer[100] = "";
            char * data = dataBuffer;
            printLine("Please enter a string: ");
            if (fgets(data, 100, stdin) < 0)
            {
                printLine("fgets failed!");
                exit(1);
            }
            printLine(data);
        }
    }
}
```

运行结果：运行

test06\CWE253_Incorrect_Check_of_Function_Return_Value__char_fgets_13.c:

```
D:\SoftwareSecurity\homewo x + v - □ x
Calling good()...
Please enter a string:
bupt
bupt

Please enter a string:
bupt
bupt

Finished good()
Calling bad()...
Please enter a string:
bupt
bupt

Finished bad()

Process returned 0 (0x0)   execution time : 8.620 s
Press any key to continue.
|
```

(1) 讯飞星火

测试输入：



测试输出：

阅书万卷，我是你的知识宝典

讯飞星火

虚拟人数字人 新建群聊 新建智能体

在检查代码时，我注意到以下潜在的问题：

1. 缓冲区溢出: fgets(data, 100, stdin) 读取最多 99 个字符（包括终止符），但 dataBuffer 的大小是 100，如果输入的字符串长度超过 99 个字符，会导致缓冲区溢出。

2. 未处理换行符: fgets 会将换行符包含在读取的字符串中，这可能会导致输出格式问题。

3. 全局常量未定义: GLOBAL_CONST_FIVE 未定义，假设它是一个全局常量，但在代码中没有提供其定义。

以下是修复后的代码：

c 复制代码

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define GLOBAL_CONST_FIVE 5 // 假设这是全局常量的定义

void function()
{
    if (GLOBAL_CONST_FIVE == 5)
    {
        char dataBuffer[100] = "";
```

代码纠错 内容写作 文本润色 网页摘要 中英翻译 学习计划 居家健身 儿童教育 更多 全新对话

点这里，尽管问，Shift+Enter 换行

内容由讯飞星火大模型生成，仅供参考

阅书万卷，我是你的知识宝典

虚拟人数字人 新建群聊 新建智能体

```
#define GLOBAL_CONST_FIVE 5 // 假设这是全局常量的定义

void function()
{
    if (GLOBAL_CONST_FIVE == 5)
    {
        char dataBuffer[100] = "";
        char *data = dataBuffer;
        printf("Please enter a string: ");
        if (fgets(data, sizeof(dataBuffer), stdin) == NULL)
        {
            printf("fgets failed!");
            exit(1);
        }

        // 移除换行符
        size_t len = strlen(data);
        if (len > 0 && data[len - 1] == '\n')
        {
            data[len - 1] = '\0';
        }

        printf(data);
    }
}
```

代码纠错 内容写作 文本润色 网页摘要 中英翻译 学习计划 居家健身 儿童教育 更多 全新对话

点这里，尽管问，Shift+Enter 换行

内容由讯飞星火大模型生成，仅供参考

60 / 113



结果分析：

该模型找到了测试代码的缓冲区溢出漏洞但并未提供漏洞所在行。

模型修改的代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define GLOBAL_CONST_FIVE 5 // 假设这是全局常量的定义

void function()
{
    if (GLOBAL_CONST_FIVE == 5)
    {
        char dataBuffer[100] = "";
        char *data = dataBuffer;
        printLine("Please enter a string: ");
        if (fgets(data, sizeof(dataBuffer), stdin) == NULL)
        {
            printLine("fgets failed!");
            exit(1);
        }

        // 移除换行符
        size_t len = strlen(data);
        if (len > 0 && data[len - 1] == '\n')
        {
```

```
        data[len - 1] = '\\0';
    }

    printLine(data);
}

void printLine(const char *message)
{
    printf("%s\\n", message);
}
```

代码运行结果：运行 test06/xunfei.c:

运行结果正确。

(2) 豆包

测试输入：



测试输出：





结果分析：

该模型找到了测试代码的缺少对输入字符串长度的检查漏洞并提供了漏洞所在行：

```
if (fgets(data, 100, stdin) < 0)
```

模型修改的代码：

```
void function()
{
    if (GLOBAL_CONST_FIVE == 5)
    {
        char dataBuffer[100] = "";
        char *data = dataBuffer;
        printLine("Please enter a string: ");
        if (fgets(data, 100, stdin) == NULL)
        {
            printLine("fgets failed!");
            exit(1);
        }

        // 检查输入字符串长度，如果包含换行符，去除换行符
        size_t len = strlen(data);
        if (len > 0 && data[len - 1] == '\n')
        {
            data[len - 1] = '\0';
        }
        else
        {

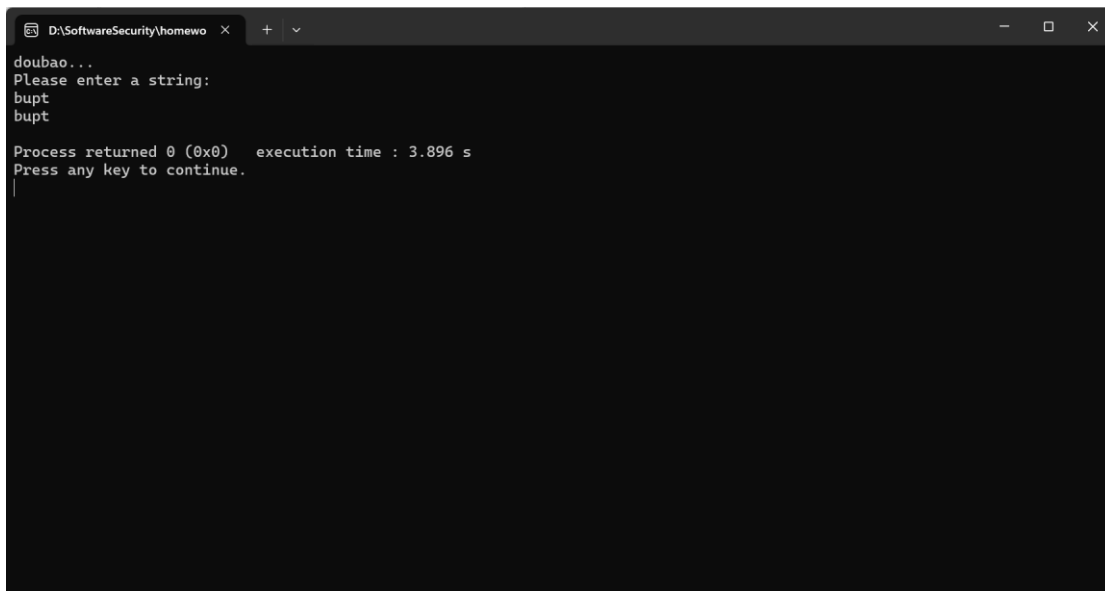
```

```
// 处理输入过长的情况，可以清空输入缓冲区
int c;
while ((c = getchar()) != '\n' && c != EOF)
    ;

printLine(data);
}
}
```

代码运行结果：运行 test06/doubao.c:

运行结果正确。



```
D:\SoftwareSecurity\homewo >
doubao...
Please enter a string:
bupt
bupt

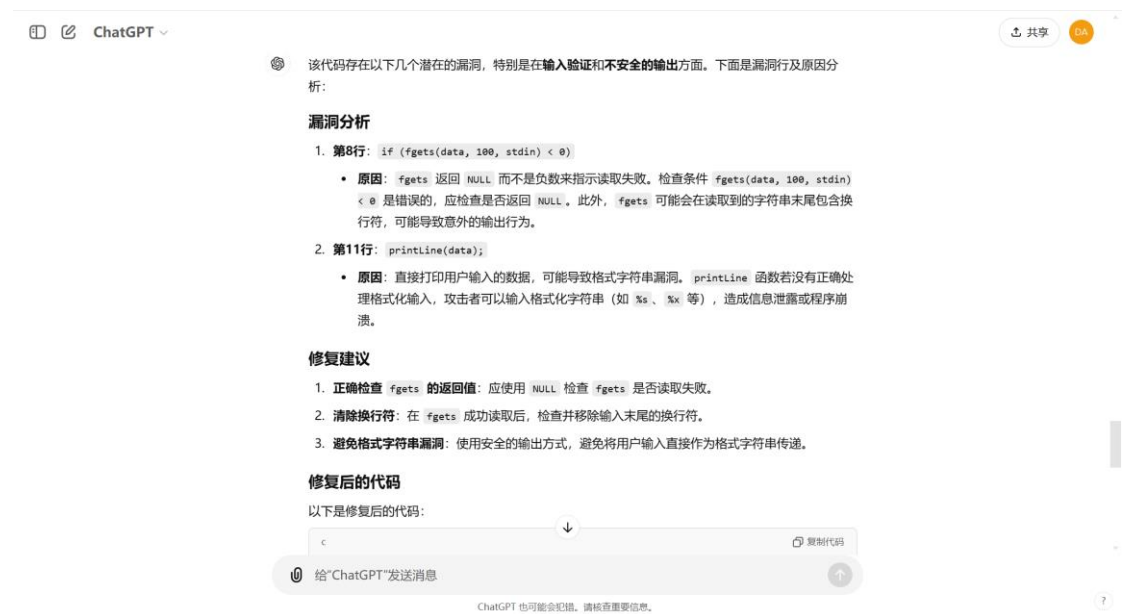
Process returned 0 (0x0)   execution time : 3.896 s
Press any key to continue.
|
```

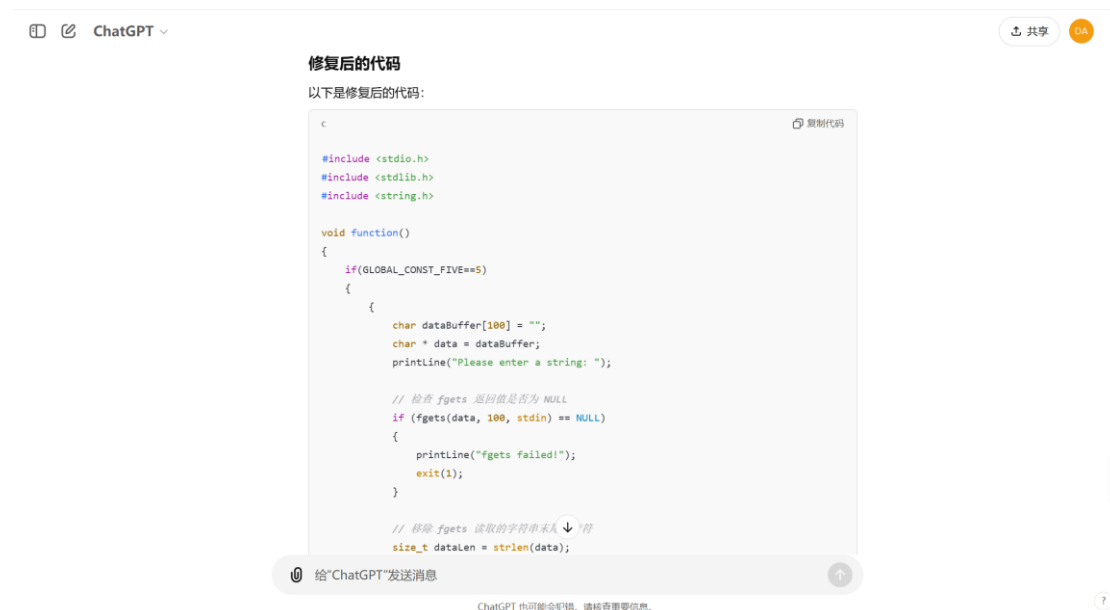
(3) ChatGPT

测试输入:



测试输出：





结果分析:

该模型找到了测试代码的输入验证和不安全的输出漏洞并提供了漏洞所在行:

```
if (fgets(data, 100, stdin) < 0)

printf(data);
```

模型修改的代码:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

void function()
{
    if(GLOBAL_CONST_FIVE==5)
    {
        {
            char dataBuffer[100] = "";
            char * data = dataBuffer;
            printLine("Please enter a string: ");

            // 检查 fgets 返回值是否为 NULL
            if (fgets(data, 100, stdin) == NULL)
            {
                printLine("fgets failed!");
                exit(1);
            }

            // 移除 fgets 读取的字符串末尾换行符
            size_t dataLen = strlen(data);
            if (dataLen > 0 && data[dataLen - 1] == '\n')
            {
                data[dataLen - 1] = '\0';
            }

            // 使用安全的输出方式，避免格式字符串漏洞
            printf("%s\n", data);
        }
    }
}

```

代码运行结果：运行 test06/chatgpt.c:

运行结果正确。


```
D:\SoftwareSecurity\homewo x + v - □ x
chatgpt...
Please enter a string:
bupt
bupt

Process returned 0 (0x0)   execution time : 2.595 s
Press any key to continue.
```

7、CWE284_Improper_Access_Control__w32_char_CreateDesktop_11.c

测试代码：

```
void function()
{
    if(globalReturnsTrue())
    {
        {
            HDESK hDesk;
            char * desktopName = "DesktopExample";
            hDesk = CreateDesktopA(
                desktopName,
                NULL,
                NULL,
                0,
                GENERIC_ALL,
                NULL);

            if (hDesk == NULL)
            {
                printLine("Desktop could not be created");
            }
            else
            {
                printLine("Desktop created successfully");
                CloseDesktop(hDesk);
            }
        }
    }
}
```

```
}
```

运行结果：运行

test07\CWE284_Improper_Access_Control__w32_char_CreateDesktop_11.c:

```
D:\SoftwareSecurity\homewo x + v
Calling good()...
Desktop could not be created
Desktop could not be created
Finished good()
Calling bad()...
Desktop created successfully
Finished bad()

Process returned 0 (0x0)   execution time : 0.099 s
Press any key to continue.
```

(1) 讯飞星火

测试输入：



测试输出：



结果分析：

该模型找到了测试代码的资源泄漏漏洞但并未提供漏洞所在行。

模型修改的代码：

```
void function()
{
    if(globalReturnsTrue())
    {
        HDESK hDesk = NULL; // 初始化为 NULL
        char * desktopName = "DesktopExample";
        hDesk = CreateDesktopA(
```

```

        desktopName,
        NULL,
        NULL,
        0,
        GENERIC_ALL,
        NULL);
if (hDesk == NULL)
{
    printLine("Desktop could not be created");
}
else
{
    printLine("Desktop created successfully");
    // 确保无论后续操作如何，都会关闭桌面句柄
    if (!CloseDesktop(hDesk))
    {
        printLine("Failed to close the desktop handle");
    }
}
}
}
}

```

代码运行结果：运行 test07/xunfei.c:

运行结果与 bad()一致。

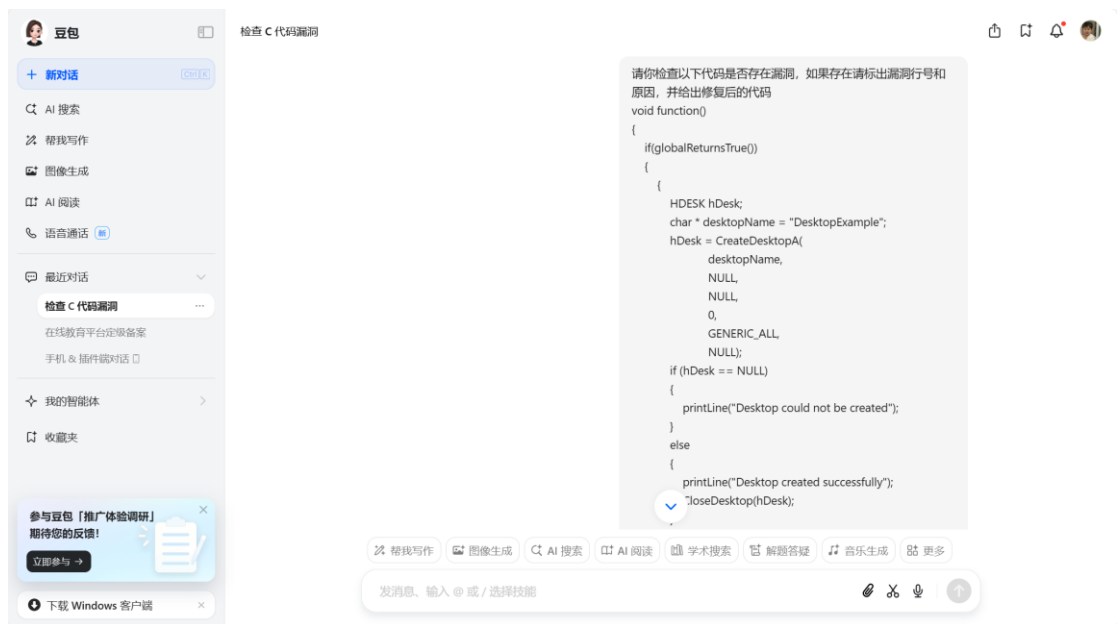
```

D:\SoftwareSecurity\homewo x + v
xunfei...
Desktop created successfully
Process returned 0 (0x0) execution time : 0.077 s
Press any key to continue.

```

(2) 豆包

测试输入：



测试输出：



结果分析：

该大模型未发现漏洞。

(3) ChatGPT

测试输入：

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    if(globalReturnsTrue())
    {
        {
            HDESK hDesk;
            char * desktopName = "DesktopExample";
            hDesk = CreateDesktopA(
                desktopName,
                NULL,
                NULL,
                0,
                GENERIC_ALL,
                NULL);
            if (hDesk == NULL)
            {
                printLine("Desktop could not be created");
            }
            else
            {
                printLine("Desktop created successfully");
                CloseDesktop(hDesk);
            }
        }
    }
}
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。

测试输出：

该代码存在以下安全性和资源管理上的漏洞，尤其在**权限控制**和**资源清理**方面存在风险。下面是漏洞行及原因分析。

漏洞分析

- 第8行: `hDesk = CreateDesktopA(desktopName, NULL, NULL, 0, GENERIC_ALL, NULL);`
 - 原因: 调用 `CreateDesktopA` 并请求 `GENERIC_ALL` 权限。这会授予该桌面所有权限，可能被恶意软件利用来执行任意代码、篡改应用程序的界面、甚至导致系统不稳定。通常不需要这么高的权限，使用最小权限原则是更安全的选择。
- 第13行: `CloseDesktop(hDesk);`
 - 原因: 如果代码中 `CloseDesktop(hDesk);` 执行失败，资源将不会正确释放，可能导致句柄泄漏。应检查 `CloseDesktop` 的返回值并在失败时适当处理，以确保资源释放。

修复建议

- 降低权限: 仅请求创建桌面所需的最小权限，以防止不必要的访问风险。
- 检查 `CloseDesktop` 的返回值: 在关闭句柄时检查其返回值，确保资源被正确释放，防止句柄泄漏。

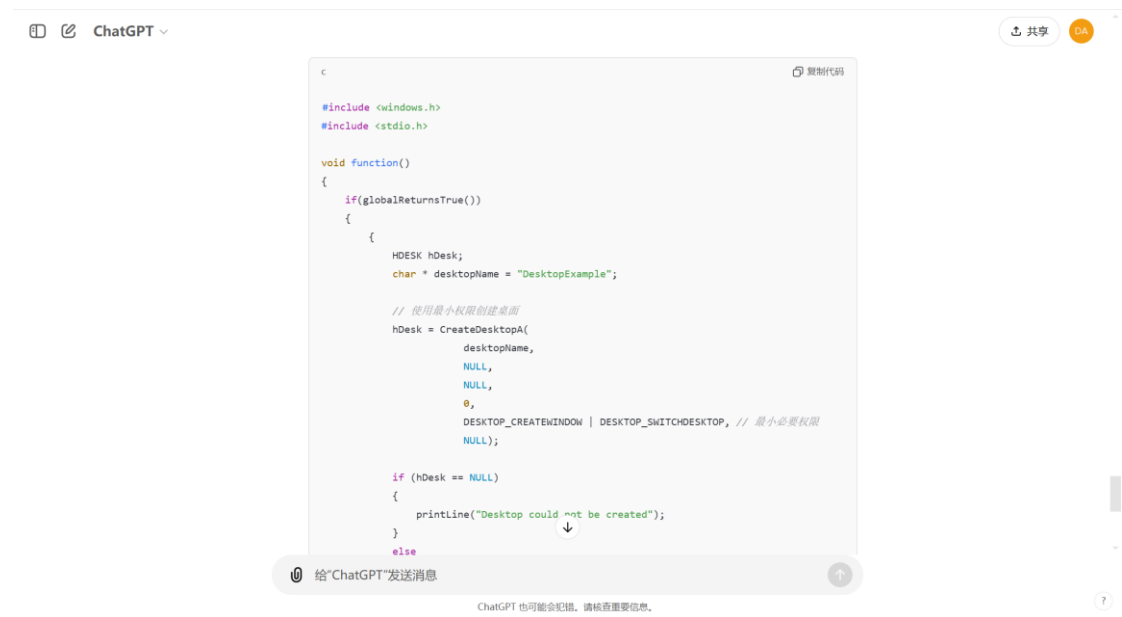
修复后的代码

以下是修复后的代码：

```
c
```

给"ChatGPT"发送消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析:

该模型找到了测试代码的权限控制和资源清理漏洞并提供了漏洞所在行:

```
hDesk = CreateDesktopA(desktopName, NULL, NULL, 0, GENERIC_ALL, NULL);  
CloseDesktop(hDesk);
```

模型修改的代码:

```
#include <windows.h>  
#include <stdio.h>  
  
void function()
```

```

{
    if(globalReturnsTrue())
    {
        {
            HDESK hDesk;
            char * desktopName = "DesktopExample";

            // 使用最小权限创建桌面
            hDesk = CreateDesktopA(
                desktopName,
                NULL,
                NULL,
                0,
                DESKTOP_CREATEWINDOW | DESKTOP_SWITCHDESKTOP, // 最小必要权限
                NULL);

            if (hDesk == NULL)
            {
                printLine("Desktop could not be created");
            }
            else
            {
                printLine("Desktop created successfully");

                // 确保句柄被正确关闭，避免泄漏
                if (!CloseDesktop(hDesk))
                {
                    printLine("Failed to close the desktop handle");
                }
            }
        }
    }
}

```

代码运行结果：运行 test07/chatgpt.c:

运行结果与 bad()一致。


```
D:\SoftwareSecurity\homewo x + v
xunfei...
Desktop created successfully

Process returned 0 (0x0)   execution time : 0.088 s
Press any key to continue.
```

8、CWE327_Use_Broken_Crypto__w32_DES_11.c

测试代码：

```
void function()
{
    if(globalReturnsTrue())
    {
        {
            FILE *pFile;
            HCRYPTPROV hCryptProv;
            HCRYPTKEY hKey;
            HCRYPTHASH hHash;
            char password[100];
            size_t passwordLen;
            char toBeDecrypted[100];
            DWORD toBeDecryptedLen = sizeof(toBeDecrypted)-1;
            printLine("Enter the password: ");
            if (fgets(password, 100, stdin) == NULL)
            {
                printLine("fgets() failed");
                password[0] = '\0';
            }
            passwordLen = strlen(password);
            if (passwordLen > 0)
            {
                password[passwordLen-1] = '\0';
            }
            pFile = fopen("encrypted.txt", "rb");
```

```

    if (pFile == NULL)
    {
        exit(1);
    }
    if (fread(toBeDecrypted, sizeof(char), 100, pFile) != 100)
    {
        fclose(pFile);
        exit(1);
    }
    toBeDecrypted[99] = '\0';
    if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES,
0))
    {
        if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV,
PROV_RSA_AES, CRYPT_NEWKEYSET))
        {
            printLine("Error in acquiring cryptographic context");
            exit(1);
        }
    }
    if(!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash))
    {
        printLine("Error in creating hash");
        exit(1);
    }
    if(!CryptHashData(hHash, (BYTE *) password, passwordLen, 0))
    {
        printLine("Error in hashing password");
        exit(1);
    }
    if(!CryptDeriveKey(hCryptProv, CALG_DES, hHash, 0, &hKey))
    {
        printLine("Error in CryptDeriveKey");
        exit(1);
    }
    if(!CryptDecrypt(hKey, 0, 1, 0, (BYTE *)toBeDecrypted, &toBeDecryptedLen))
    {
        printLine("Error in decryption");
        exit(1);
    }
    toBeDecrypted[toBeDecryptedLen] = '\0';
    printLine(toBeDecrypted);
    if (hKey)
    {

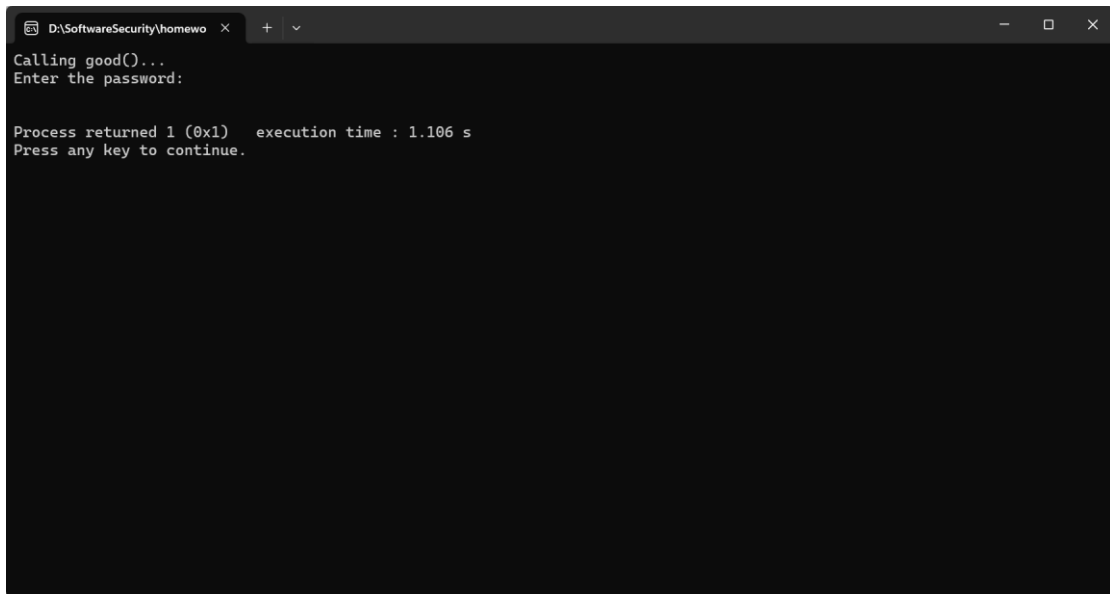
```

```

        CryptDestroyKey(hKey);
    }
    if (hHash)
    {
        CryptDestroyHash(hHash);
    }
    if (hCryptProv)
    {
        CryptReleaseContext(hCryptProv, 0);
    }
    if (pFile)
    {
        fclose(pFile);
    }
}
}
}

```

运行结果：运行 test08\CWE327_Use_Broken_Crypto__w32_DES_11.c:



```

D:\SoftwareSecurity\homewo x + v
Calling good()...
Enter the password:

Process returned 1 (0x1) execution time : 1.106 s
Press any key to continue.

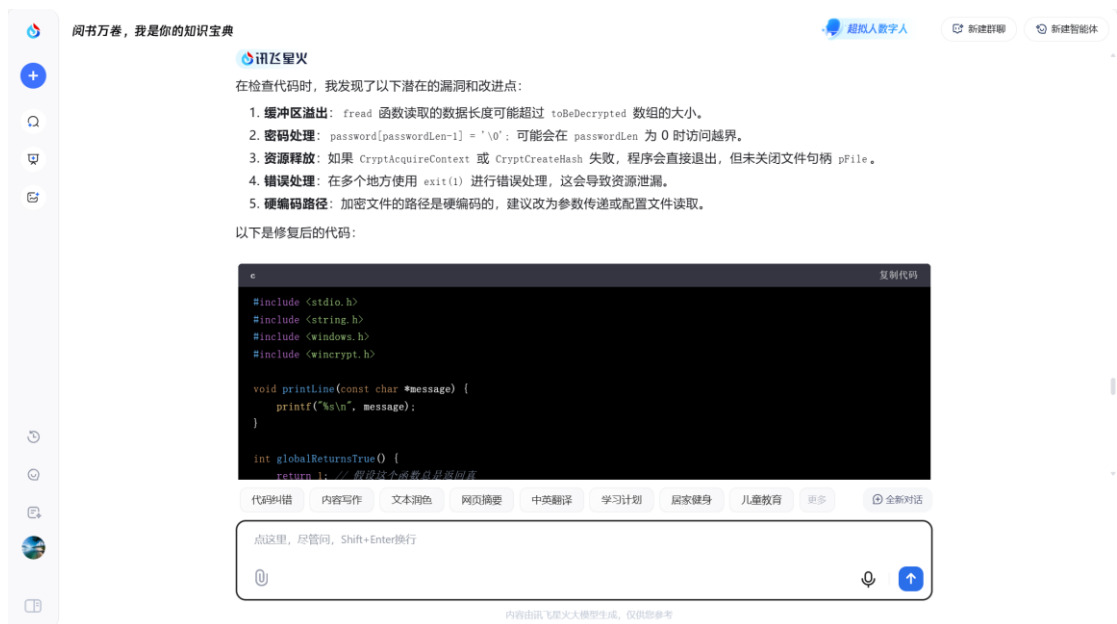
```

(1) 讯飞星火

测试输入:



测试输出：





结果分析：

该模型找到了测试代码的缓冲区溢出、密码处理、资源释放、错误处理漏洞但并未提供漏洞所在行。

正确漏洞修复方法应是用 AES 替代 DES，但该模型并未如此做。

模型修改的代码：

```
#include <stdio.h>
#include <string.h>
#include <windows.h>
#include <wincrypt.h>

void printLine(const char *message) {
    printf("%s\n", message);
}

int globalReturnsTrue() {
    return 1; // 假设这个函数总是返回真
}

void function() {
    if (globalReturnsTrue()) {
        FILE *pFile = NULL;
        HCRYPTPROV hCryptProv = 0;
        HCRYPTKEY hKey = 0;
        HCRYPTHASH hHash = 0;
```

```

char password[100];
size_t passwordLen;
char toBeDecrypted[100];
DWORD toBeDecryptedLen = sizeof(toBeDecrypted) - 1;

println("Enter the password: ");
if (fgets(password, sizeof(password), stdin) == NULL) {
    println("fgets() failed");
    password[0] = '\0';
} else {
    passwordLen = strlen(password);
    if (passwordLen > 0 && password[passwordLen - 1] == '\n') {
        password[passwordLen - 1] = '\0';
    }
}

pFile = fopen("encrypted.txt", "rb");
if (pFile == NULL) {
    println("Failed to open file");
    return;
}

if (fread(toBeDecrypted, sizeof(char), sizeof(toBeDecrypted) - 1, pFile) !=
sizeof(toBeDecrypted) - 1) {
    println("Failed to read file");
    fclose(pFile);
    return;
}
toBeDecrypted[99] = '\0';
fclose(pFile);
pFile = NULL;

if (!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES,
0)) {
    if (!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV,
PROV_RSA_AES, CRYPT_NEWKEYSET)) {
        println("Error in acquiring cryptographic context");
        return;
    }
}

if (!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash)) {
    println("Error in creating hash");
    CryptReleaseContext(hCryptProv, 0);
}

```

```

        return;
    }

    if (!CryptHashData(hHash, (BYTE *)password, strlen(password), 0)) {
        printLine("Error in hashing password");
        CryptDestroyHash(hHash);
        CryptReleaseContext(hCryptProv, 0);
        return;
    }

    if (!CryptDeriveKey(hCryptProv, CALG_DES, hHash, 0, &hKey)) {
        printLine("Error in CryptDeriveKey");
        CryptDestroyHash(hHash);
        CryptReleaseContext(hCryptProv, 0);
        return;
    }

    if (!CryptDecrypt(hKey, 0, 1, 0, (BYTE *)toBeDecrypted, &toBeDecryptedLen)) {
        printLine("Error in decryption");
        CryptDestroyKey(hKey);
        CryptDestroyHash(hHash);
        CryptReleaseContext(hCryptProv, 0);
        return;
    }
    toBeDecrypted[toBeDecryptedLen] = '\0';
    printLine(toBeDecrypted);

    if (hKey) {
        CryptDestroyKey(hKey);
    }
    if (hHash) {
        CryptDestroyHash(hHash);
    }
    if (hCryptProv) {
        CryptReleaseContext(hCryptProv, 0);
    }
}
}

```

代码运行结果：运行 test08/xunfei.c:

```
D:\SoftwareSecurity\homewo x + v
xunfei...
Enter the password:
Failed to read file

Process returned 0 (0x0)   execution time : 1.169 s
Press any key to continue.
```

(2) 豆包

测试输入:



测试输出:



结果分析：

该模型找到了测试代码的密码以明文形式处理和缓冲区溢出漏洞并提供了漏洞所在行：

整个函数。

正确漏洞修复方法应是用 AES 替代 DES，但该模型并未如此做。

模型修改的代码：

```
void function()  
{
```

```

if (globalReturnsTrue())
{
    FILE *pFile;
    HCRYPTPROV hCryptProv;
    HCRYPTKEY hKey;
    HCRYPTHASH hHash;
    char password[100];
    size_t passwordLen;
    // 不再使用固定长度的缓冲区来存储待解密数据
    char *toBeDecrypted = NULL;
    DWORD toBeDecryptedLen = 0;

    printLine("Enter the password: ");
    if (fgets(password, 100, stdin) == NULL)
    {
        printLine("fgets() failed");
        password[0] = '\0';
    }
    passwordLen = strlen(password);
    if (passwordLen > 0)
    {
        password[passwordLen - 1] = '\0';

        // 使用密码安全的哈希算法库来处理密码
        // 这里只是示例，实际应用中需要正确集成和使用密码哈希库
        // 假设经过密码哈希库处理后的密码哈希存储在 passwordHash 中
        // char passwordHash[HASH_LENGTH];
        // hashPassword(password, passwordHash);
    }

    pFile = fopen("encrypted.txt", "rb");
    if (pFile == NULL)
    {
        exit(1);
    }

    // 获取文件大小
    fseek(pFile, 0, SEEK_END);
    long fileSize = ftell(pFile);
    fseek(pFile, 0, SEEK_SET);

    // 根据文件大小分配内存
    toBeDecrypted = (char *)malloc(fileSize + 1);
    if (toBeDecrypted == NULL)

```

```

    {
        fclose(pFile);
        exit(1);
    }

    // 读取文件数据
    if (fread(toBeDecrypted, sizeof(char), fileSize, pFile) != fileSize)
    {
        free(toBeDecrypted);
        fclose(pFile);
        exit(1);
    }
    toBeDecrypted[fileSize] = '\0';
    toBeDecryptedLen = fileSize;

    if (!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES,
0))
    {
        if (!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV,
PROV_RSA_AES, CRYPT_NEWKEYSET))
        {
            printLine("Error in acquiring cryptographic context");
            free(toBeDecrypted);
            exit(1);
        }
    }

    if (!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash))
    {
        printLine("Error in creating hash");
        free(toBeDecrypted);
        exit(1);
    }

    if (!CryptHashData(hHash, (BYTE *)password, passwordLen, 0))
    {
        printLine("Error in hashing password");
        free(toBeDecrypted);
        exit(1);
    }

    if (!CryptDeriveKey(hCryptProv, CALG_DES, hHash, 0, &hKey))
    {
        printLine("Error in CryptDeriveKey");
    }

```

```

        free(toBeDecrypted);
        exit(1);
    }

    if (!CryptDecrypt(hKey, 0, 1, 0, (BYTE *)toBeDecrypted, &toBeDecryptedLen))
    {
        printLine("Error in decryption");
        free(toBeDecrypted);
        exit(1);
    }

    printLine(toBeDecrypted);

    if (hKey)
    {
        CryptDestroyKey(hKey);
    }
    if (hHash)
    {
        CryptDestroyHash(hHash);
    }
    if (hCryptProv)
    {
        CryptReleaseContext(hCryptProv, 0);
    }
    if (pFile)
    {
        fclose(pFile);
    }

    // 释放分配的内存
    free(toBeDecrypted);
}
}

```

代码运行结果：运行 test08/doubao.c:

```
D:\SoftwareSecurity\homewo x + v
doubao...
Enter the password:
Error in decryption
Process returned 1 (0x1) execution time : 1.454 s
Press any key to continue.
```

(3) ChatGPT

测试输入:



测试输出:

此代码存在以下漏洞，主要涉及敏感信息清理、错误处理和资源管理。下面是漏洞行及原因分析。

漏洞分析

- 第14行:** `fgets(password, 100, stdin);`
 - 原因:** 从标准输入读取密码后未清理，释放资源后可能在内存中残留敏感信息，导致密码泄露风险。
- 第24行:** `if (fread(toBeDecrypted, sizeof(char), 100, pFile) != 100)`
 - 原因:** `fread` 未正确读取到 100 字节数据时直接退出，未考虑文件可能小于 100 字节。此操作会导致未完整读取内容的情况下程序退出，可能影响程序逻辑。
- 第29行:** `CryptAcquireContext`
 - 原因:** `CryptAcquireContext` 和后续的加密函数未考虑资源的分配与释放顺序，一旦函数执行失败会直接 `exit(1)`，可能导致句柄泄漏。此外，若无法获取上下文，系统会尝试创建新密钥集，但可能没有必要。
- 缺少密码清理:** 敏感数据 `password` 和 `toBeDecrypted` 在使用完成后应当被安全清理，避免内存中保留明文信息。

修复建议

- 清理敏感数据:** 在释放资源和退出前，使用 `memset` 清理 `password` 和 `toBeDecrypted` 的内容。
- 文件读取的长度检查:** 在读取文件时动态计算数据长度，避免固定长度读取的潜在错误。

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请检查重要信息。

修复建议

- 清理敏感数据:** 在释放资源和退出前，使用 `memset` 清理 `password` 和 `toBeDecrypted` 的内容。
- 文件读取的长度检查:** 在读取文件时动态计算数据长度，避免固定长度读取的潜在错误。
- 改进错误处理与资源释放:** 确保每个加密函数失败时释放已分配的资源。

修复后的代码

以下是改进后的代码：

```
c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void function()
{
    if(globalReturnsTrue())
    {
        FILE *pFile = NULL;
        HCRYPTPROV hCryptProv = 0;
        HCRYPTKEY hKey = 0;
        HCRYPTHASH hHash = 0;
        char password[100];
    }
}
```

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请检查重要信息。



结果分析：

该模型找到了测试代码的敏感信息清理、错误处理和资源管理漏洞并提供了漏洞所在行：

<code>fgets(password, 100, stdin);</code>
<code>if (fread(toBeDecrypted, sizeof(char), 100, pFile) != 100)</code>
<code>CryptAcquireContext</code>

正确漏洞修复方法应是用 AES 替代 DES，但该模型并未如此做。

模型修改的代码：

<pre>#include <windows.h> #include <stdio.h> #include <stdlib.h> #include <string.h> void function() { if(globalReturnsTrue()) { FILE *pFile = NULL; HCRYPTPROV hCryptProv = 0; HCRYPTKEY hKey = 0; HCRYPTHASH hHash = 0; char password[100]; size_t passwordLen;</pre>
--

```

char toBeDecrypted[100] = {0}; // 初始化为零
DWORD toBeDecryptedLen = sizeof(toBeDecrypted)-1;

printLine("Enter the password: ");
if (fgets(password, 100, stdin) == NULL)
{
    printLine("fgets() failed");
    password[0] = '\0';
    return;
}

// 去除末尾换行符
passwordLen = strlen(password);
if (passwordLen > 0)
{
    password[passwordLen-1] = '\0';
}

pFile = fopen("encrypted.txt", "rb");
if (pFile == NULL)
{
    printLine("File open failed.");
    memset(password, 0, sizeof(password)); // 清理密码
    return;
}

size_t bytesRead = fread(toBeDecrypted, sizeof(char), sizeof(toBeDecrypted)-1,
pFile);
if (bytesRead == 0)
{
    printLine("File read failed.");
    fclose(pFile);
    memset(password, 0, sizeof(password)); // 清理密码
    return;
}
toBeDecrypted[bytesRead] = '\0';

if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES, 0))
{
    if(!CryptAcquireContext(&hCryptProv, NULL, MS_ENH_RSA_AES_PROV, PROV_RSA_AES,
CRYPT_NEWKEYSET))
    {
        printLine("Error in acquiring cryptographic context");
        memset(password, 0, sizeof(password)); // 清理密码
    }
}

```



```

        fclose(pFile);
        return;
    }
}

if(!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash))
{
    printLine("Error in creating hash");
    memset(password, 0, sizeof(password));
    CryptReleaseContext(hCryptProv, 0);
    fclose(pFile);
    return;
}

if(!CryptHashData(hHash, (BYTE *) password, passwordLen, 0))
{
    printLine("Error in hashing password");
    memset(password, 0, sizeof(password));
    CryptDestroyHash(hHash);
    CryptReleaseContext(hCryptProv, 0);
    fclose(pFile);
    return;
}

if(!CryptDeriveKey(hCryptProv, CALG_DES, hHash, 0, &hKey))
{
    printLine("Error in CryptDeriveKey");
    memset(password, 0, sizeof(password));
    CryptDestroyHash(hHash);
    CryptReleaseContext(hCryptProv, 0);
    fclose(pFile);
    return;
}

if(!CryptDecrypt(hKey, 0, TRUE, 0, (BYTE *)toBeDecrypted, &toBeDecryptedLen))
{
    printLine("Error in decryption");
    memset(password, 0, sizeof(password));
    memset(toBeDecrypted, 0, sizeof(toBeDecrypted));
    CryptDestroyKey(hKey);
    CryptDestroyHash(hHash);
    CryptReleaseContext(hCryptProv, 0);
    fclose(pFile);
    return;
}

```

```

    }
    toBeDecrypted[toBeDecryptedLen] = '\0';
    printLine(toBeDecrypted);

    // 清理敏感数据并释放资源
    memset(password, 0, sizeof(password));
    memset(toBeDecrypted, 0, sizeof(toBeDecrypted));
    if (hKey)
    {
        CryptDestroyKey(hKey);
    }
    if (hHash)
    {
        CryptDestroyHash(hHash);
    }
    if (hCryptProv)
    {
        CryptReleaseContext(hCryptProv, 0);
    }
    if (pFile)
    {
        fclose(pFile);
    }
}
}

```

代码运行结果：运行 test08/chatgpt.c:

```

D:\SoftwareSecurity\homewo x + v
chatgpt...
Enter the password:
Error in decryption
Process returned 0 (0x0) execution time : 1.046 s
Press any key to continue.

```

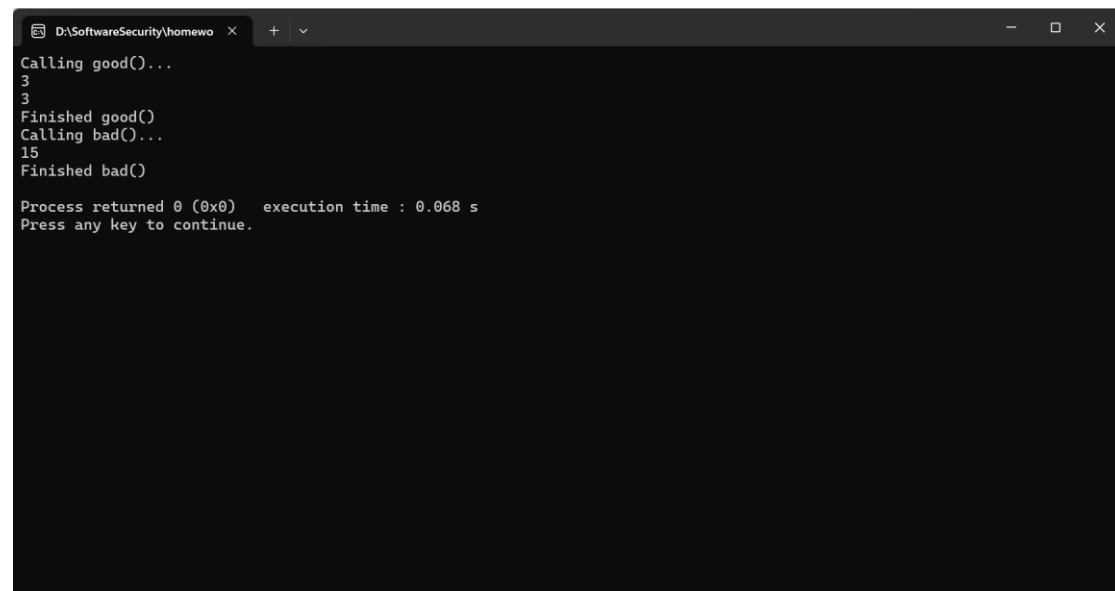
9、CWE469_Use_of_Pointer_Subtraction_to_Determine_Size__char_14.c

测试代码：

```
void function()
{
    if(globalFive==5)
    {
        {
            char string1[] = SOURCE_STRING;
            char string2[] = SOURCE_STRING;
            char * slashInString1;
            size_t indexOfSlashInString1;
            slashInString1 = strchr(string1, '/');
            if (slashInString1 == NULL)
            {
                exit(1);
            }
            indexOfSlashInString1 = (size_t)(slashInString1 - string2);
            printUnsignedLine(indexOfSlashInString1);
        }
    }
}
```

运行结果：运行

test09/CWE469_Use_of_Pointer_Subtraction_to_Determine_Size__char_14.c:



```
D:\SoftwareSecurity\homewo x + -
Calling good()...
3
3
Finished good()
Calling bad()...
15
Finished bad()

Process returned 0 (0x0)   execution time : 0.068 s
Press any key to continue.
```

(1) 讯飞星火

测试输入：



测试输出：





结果分析：

该模型找到了测试代码的缓冲区溢出漏洞但并未提供漏洞所在行。

模型修改的代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

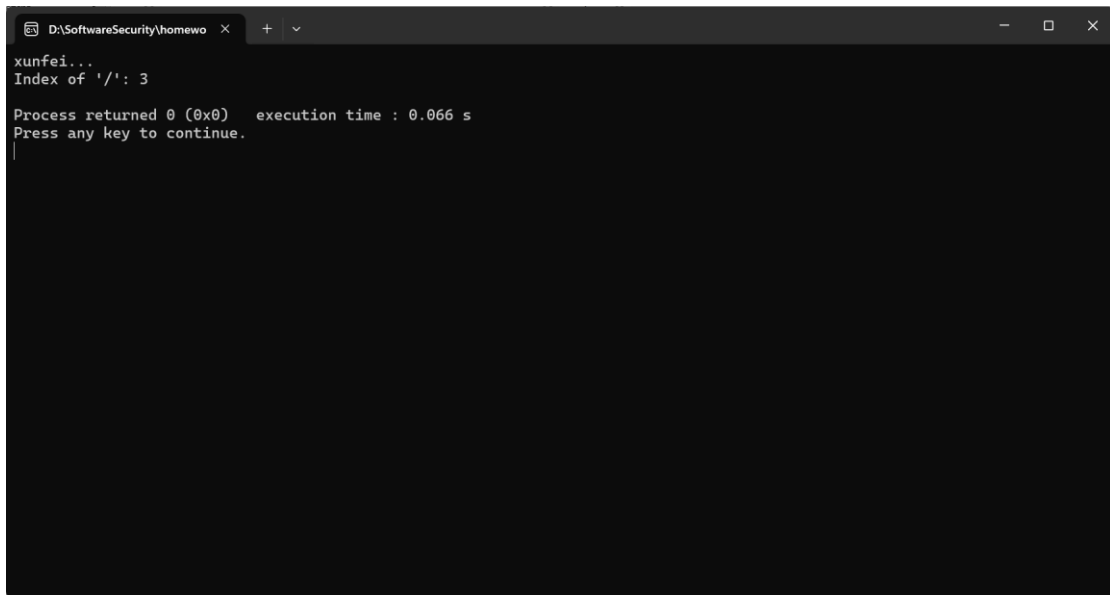
#define SOURCE_STRING "example/source/string"

void function()
{
    int globalFive = 5; // 假设 globalFive 被定义为 5
    if (globalFive == 5)
    {
        char string1[] = SOURCE_STRING;
        char *slashInString1;
        size_t indexOfSlashInString1;
        slashInString1 = strchr(string1, '/');
        if (slashInString1 == NULL)
        {
            fprintf(stderr, "Error: '/' not found in the source string.\n");
            exit(EXIT_FAILURE); // 使用标准退出码
        }
        indexOfSlashInString1 = (size_t)(slashInString1 - string1); // 修正为 string1
        printf("Index of '/': %zu\n", indexOfSlashInString1); // 使用 printf 代替
        printUnsignedLine
    }
}
```

```
    }  
}  
  
int main() {  
    function();  
    return 0;  
}
```

代码运行结果：运行 test09/xunfei.c:

运行结果正确，与 good()一致。



```
D:\SoftwareSecurity\homewo x + v  
xunfei...  
Index of '/': 3  
Process returned 0 (0x0) execution time : 0.066 s  
Press any key to continue.
```

(2) 豆包

测试输入:



测试输出：



结果分析：

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行：第 5 行和第 6 行。

模型修改的代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SOURCE_STRING "some/example/string"
```

```

void printUnsignedLine(size_t num);

void function() {
    int globalFive = 5;
    if (globalFive == 5) {
        char string1[100];
        char string2[100];
        strcpy(string1, SOURCE_STRING);
        strcpy(string2, SOURCE_STRING);
        char *slashInString1;
        size_t indexOfSlashInString1;
        slashInString1 = strchr(string1, '/');
        if (slashInString1 == NULL) {
            exit(1);
        }
        indexOfSlashInString1 = (size_t)(slashInString1 - string2);
        printUnsignedLine(indexOfSlashInString1);
    }
}
}

```

代码运行结果：运行 test09/doubao.c:

运行结果错误。

```

D:\SoftwareSecurity\homewo x + v
doubao...
4294967187
Process returned 0 (0x0)   execution time : 0.064 s
Press any key to continue.

```

(3) ChatGPT

测试输入:

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    if(globalFive==5)
    {
        {
            char string1[] = SOURCE_STRING;
            char string2[] = SOURCE_STRING;
            char * slashInString1;
            size_t indexOfSlashInString1;
            slashInString1 = strchr(string1, '/');
            if (slashInString1 == NULL)
            {
                exit(1);
            }
            indexOfSlashInString1 = (size_t)(slashInString1 - string2);
            printUnsignedLine(indexOfSlashInString1);
        }
    }
}
```

在这段代码中，存在一个漏洞：

漏洞行号：

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请检查重要信息。

测试输出：

```
}
}
```

在这段代码中，存在一个漏洞：

漏洞行号：

```
c
indexOfSlashInString1 = (size_t)(slashInString1 - string2);
```

复制代码

原因：在这一行，`slashInString1` 是在 `string1` 中找到的字符指针，而 `string2` 是一个不同的字符数组。因此，`slashInString1 - string2` 的计算在两个不同的数组之间进行，导致未定义行为。这可能会引发程序崩溃或出现不可预知的错误。

修复后的代码：为了修复这个漏洞，应使用 `slashInString1` 和 `string1` 的基地址进行指针偏移运算。这样可以确保计算的索引是基于同一个数组的：

```
c
void function()
{
    if(globalFive == 5)
    {
        {
            char string1[] = SOURCE_STRING;
```

复制代码

给“ChatGPT”发送消息

ChatGPT 也可能犯错。请检查重要信息。



结果分析：

该模型找到了测试代码的缓冲区溢出漏洞并提供了漏洞所在行：

```
indexOfSlashInString1 = (size_t)(slashInString1 - string2);
```

模型修改的代码：

```
void function()
{
    if(globalFive == 5)
    {
        {
            char string1[] = SOURCE_STRING;
            char string2[] = SOURCE_STRING;
            char *slashInString1;
            size_t indexOfSlashInString1;
            slashInString1 = strchr(string1, '/');
            if (slashInString1 == NULL)
            {
                exit(1);
            }
            indexOfSlashInString1 = (size_t)(slashInString1 - string1); // 修复行
            printUnsignedLine(indexOfSlashInString1);
        }
    }
}
```

代码运行结果：运行 test09/chatgpt.c：

运行结果正确，与 good()一致。

A screenshot of a Windows command prompt window. The title bar shows the path 'D:\SoftwareSecurity\homewo'. The command prompt displays the output of a program: 'chatgpt...' followed by '3', then 'Process returned 0 (0x0) execution time : 0.067 s' and 'Press any key to continue.' The window is dark-themed.

10、CWE511_Logic_Time_Bomb__counter_16.c

测试代码：

```
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
                if (count == COUNT_CHECK)
                {
                    UNLINK("important_file.txt");
                }
                count++;
            }
            while(1 == 1);
        }
        break;
    }
}
```

 important_file1.txt	2024/11/12 14:18	文本文档	0 KB
 important_file2.txt	2024/11/12 14:18	文本文档	0 KB
 important_file3.txt	2024/11/12 14:18	文本文档	0 KB
 important_file4.txt	2024/11/12 14:18	文本文档	0 KB

运行结果：运行 test10/ CWE511_Logic_Time_Bomb__counter_16.c：

important_file1.txt 已被删除。

 important_file2.txt	2024/11/12 14:18	文本文档	0 KB
 important_file3.txt	2024/11/12 14:18	文本文档	0 KB
 important_file4.txt	2024/11/12 14:18	文本文档	0 KB

(1) 讯飞星火

测试输入：



测试输出：

阅书万卷，我是你的知识宝典

讯飞星火

超拟人数字人

新建群聊

新建智能体

+

🔍

📄

📧

🕒

🔄

📁

🌐

📖

讯飞星火

这段代码存在多个问题，包括潜在的无限循环、未定义的宏以及可能的文件删除操作。以下是详细的分析和修复建议：

问题分析

1. 无限循环:

- while(1 == 1): 是一个永远为真的条件，导致内部的 do-while 循环永远不会终止。
- 外层的 while(1) 也是一个无限循环，因为内部有一个 break 语句，但这个 break 只会在第一次迭代后执行，因此整个函数会陷入一个无限循环。

2. 未定义的宏:

- COUNT_CHECK 和 UNLINK 这两个宏没有定义，可能会导致编译错误或运行时错误。

3. 文件删除操作:

- UNLINK("important_file.txt"): 直接删除了一个重要文件，这通常是不安全的操作，除非有非常明确的理由和适当的权限检查。

修复后的代码

假设 COUNT_CHECK 是一个预定义的常量，并且 UNLINK 是一个合法的文件删除函数（例如标准库中的 unlink），我们可以对代码进行如下修复：

代码纠错

内容写作

文本润色

网页摘要

中英翻译

学习计划

居家健身

儿童教育

更多

🗨️ 全新对话

点这里，尽管问，Shift+Enter换行

🗨️

🔊

📤

内容由讯飞星火大模型生成，仅供参考

阅书万卷，我是你的知识宝典

讯飞星火

超拟人数字人

新建群聊

新建智能体

+

🔍

📄

📧

🕒

🔄

📁

🌐

📖

讯飞星火

修复后的代码

假设 COUNT_CHECK 是一个预定义的常量，并且 UNLINK 是一个合法的文件删除函数（例如标准库中的 unlink），我们可以对代码进行如下修复：

代码纠错

内容写作

文本润色

网页摘要

中英翻译

学习计划

居家健身

儿童教育

更多

🗨️ 全新对话

点这里，尽管问，Shift+Enter换行

🗨️

🔊

📤

内容由讯飞星火大模型生成，仅供参考

105 / 113



结果分析：

该模型找到了测试代码的缓冲区溢出漏洞但并未提供漏洞所在行。

模型修改的代码：

```
#include <stdio.h>
#include <unistd.h> // for unlink function

#define COUNT_CHECK 1000 // Example value, should be defined appropriately

void function()
{
    int count = 0;
    while (count < COUNT_CHECK)
    {
        // Perform some operations here if needed
        count++;
    }

    // After the loop, delete the file
    if (unlink("important_file.txt") != 0)
    {
        perror("Error deleting file");
    }
}
```

代码运行结果：运行 test10/xunfei.c:

important_file2.txt 已被删除。

 important_file3.txt	2024/11/12 14:18	文本文档	0 KB
 important_file4.txt	2024/11/12 14:18	文本文档	0 KB

(2) 豆包

测试输入：



测试输出：



结果分析：

该模型找到了测试代码的无限循环漏洞并提供了漏洞所在行：第 7 行。

模型修改的代码：

```
#include <stdio.h>
#include <stdlib.h>

#define COUNT_CHECK 5
void unlinkFunction(const char *filename);

void function() {
    while (1) {
        {
            int count = 0;
            do {
                if (count == COUNT_CHECK) {
                    unlinkFunction("important_file.txt");
                }
                count++;
            } while (count < 10);
        }
        break;
    }
}

void unlinkFunction(const char *filename) {
    if (remove(filename) != 0) {
        perror("Error unlinking file");
    }
}
```

代码运行结果：运行 test10/doubao.c:

important_file3.txt 未被删除。

 important_file3.txt	2024/11/12 14:18	文本文档	0 KB
 important_file4.txt	2024/11/12 14:18	文本文档	0 KB

(3) ChatGPT

测试输入：

请你检查以下代码是否存在漏洞，如果存在请标出漏洞行号和原因，并给出修复后的代码

```
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
                if (count == COUNT_CHECK)
                {
                    UNLINK("important_file.txt");
                }
                count++;
            }
            while(1 == 1);
        }
        break;
    }
}
```

这段代码中存在一个漏洞：

给“ChatGPT”发消息

ChatGPT 也可能犯错。请核查重要信息。

测试输出：

这段代码中存在一个漏洞：

漏洞行号：

```
C
while(1 == 1);
```

原因： 这里的 `while(1 == 1);` 形成了一个无限循环，因为 `1 == 1` 始终为真，这会导致程序在该循环中不断执行下去。在 `count` 达到 `COUNT_CHECK` 时将会调用 `UNLINK("important_file.txt")` 删除文件 `important_file.txt`。由于 `count` 不会被重置，此循环会继续删除文件而不停止，可能导致文件意外删除或资源浪费。

修复后的代码： 应修改 `while(1 == 1)` 以提供适当的循环退出条件，例如设定上限或根据某种条件终止循环。以下代码示例使用 `break` 来退出循环：

```
C
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
```

给“ChatGPT”发消息

ChatGPT 也可能犯错。请核查重要信息。



结果分析:

该模型找到了测试代码的无限循环漏洞并提供了漏洞所在行:

```
while(1 == 1);
```

模型修改的代码:

```
void function()
{
    while(1)
    {
        {
            int count = 0;
            do
            {
                if (count == COUNT_CHECK)
                {
                    UNLINK("important_file.txt");
                    break; // 修复行: 退出循环
                }
                count++;
            }
            while(count < COUNT_CHECK + 1); // 修复行: 限定循环条件
        }
        break;
    }
}
```

代码运行结果：运行 test10/chatgpt.c:

important_file4.txt 已被删除。

 important_file3.txt

2024/11/12 14:18

文本文档

0 KB

二、分析

测试用例	模型	是否找到代码漏洞	是否提供漏洞行数	是否提供漏洞原因	代码是否运行	漏洞是否修复	备注
TEST01	讯飞星火	是	是	是	是	是	
	豆包	是	是	是	是	是	
	ChatGPT	是	是	是	是	是	
TEST02	讯飞星火	是	是	是	是	是	
	豆包	是	是	是	否		
	ChatGPT	是	是	是	否		
TEST03	讯飞星火	是	否	是	否		
	豆包	是	是	是	是	否	输出结果与 bad()一致，与 good()不一致，说明并未修复漏洞
	ChatGPT	是	是	是	是	否	输出结果与 bad()一致，与 good()不一致，说明并未修复漏洞
TEST04	讯飞星火	是	是	是	是	是	
	豆包	是	是	是	是	是	
	ChatGPT	是	是	是	是	是	
TEST05	讯飞星火	是	否	是	是	是	

	豆包	是	是	是	是	是	
	ChatGPT	是	是	是	是	是	
TEST06	讯飞星火	是	否	是	是	是	
	豆包	是	是	是	是	是	
	ChatGPT	是	是	是	是	是	
TEST07	讯飞星火	是	否	是	是	否	输出结果与 bad() 一致，与 good() 不一致，说明并未修复漏洞
	豆包	否					
	ChatGPT	是	是	是	是	否	输出结果与 bad() 一致，与 good() 不一致，说明并未修复漏洞
TEST08	讯飞星火	是	否	是	是	否	正确漏洞修复方法应是用 AES 替代 DES，但模型并未如此做
	豆包	是	是	是	是	否	正确漏洞修复方法应是用 AES 替代 DES，但模型并未如此做
	ChatGPT	是	是	是	是	否	正确漏洞修复方法应是用 AES 替代 DES，但模型并未如此做
TEST09	讯飞星火	是	否	是	是	是	
	豆包	是	是	是	是	否	输出奇怪的错误结果，说明并未修复漏洞
	ChatGPT	是	是	是	是	是	
TEST10	讯飞星火	是	否	是	是	是	

豆包	是	是	是	是	否	代码错误运行， important_file3.txt 未被 删除
ChatGPT	是	是	是	是	是	

在漏洞检测方面：讯飞星火和 ChatGPT 能识别所有测试用例的漏洞，而豆包却有一个测试用例并未发现漏洞；然而讯飞星火在提供漏洞行号这一点的表现却十分不稳定，大部分的用例无法提供行号，其余的两个模型均能正确提供行号；三个模型均能分析出漏洞原因，但仔细阅读所分析的原因，在个别测试用例中，其分析的原因并不十分准确，而是给人一种“胡编乱造”的感觉，有意思的是，两个国产大模型似乎很热衷于缓冲区溢出漏洞，尤其是讯飞星火，近一半的测试用例都给出了缓冲区溢出漏洞。

在漏洞修复方面：三个模型均能对漏洞进行修复，但修复结果却显得不尽如人意。三个模型均是有一个用例不能正确运行，然而对于个别测试用例来说，尽管代码能正确运行，但并没有对漏洞进行有效修复，尤其是豆包，有将近一半（4/9）的测试用例都没有进行正确修复，表现较差。

综上所述，综合考量漏洞检测和漏洞修复两个方面，ChatGPT 的表现最好，但在复杂安全修复逻辑上仍未完全成熟，仍有较大改进空间。